

SHiP SOFTWARE

HANDS-ON TUTORIAL

The SHiP Software Framework

From a heavy neutral lepton to digitised hits

Tutorial booklet

Participant edition

Geometry • geometry and field services

Data model • aegir • sea-cucumber • Shannon

M.Climescu

matclim@cern.ch

October 17, 2026

Software repositories:
`https://github.com/ShipSoft`

Reviewed draft. The environment and input-sample details marked **TBD** must be supplied before the hands-on session.

Contents

How to use this booklet vi

I. Working together 1

1. Git on your own 2
1.1. What git keeps . . . 2
1.2. First steps . . . 3
1.3. Good commits . . . 3
1.4. What not to commit . . . 4

2. Branches: merging and rebasing 5
2.1. Branches . . . 5
2.2. Merging and rebasing . . . 5
2.3. Conflicts . . . 6

3. Remotes and pull requests 8
3.1. Remotes . . . 8
3.2. Forks . . . 9
3.3. Pull requests the ShipSoft way . . . 9
3.4. Git cheat sheet . . . 11

4. Reproducible environments with pixi 12
4.1. Why an environment, and why pixi . . . 12
4.2. Installing pixi . . . 12
4.3. Your first workspace . . . 13
4.4. Tasks . . . 13
4.5. More than one environment . . . 14
4.6. Keeping everybody on the same versions . . . 14
4.7. How the ShipSoft repositories use pixi . . . 15

5. Automatic checks: continuous integration 16
5.1. What CI does . . . 16
5.2. GitHub Actions in five words . . . 16
5.3. Your first workflow . . . 17
5.4. The workflow of the team repository . . . 18
5.5. CI in the ShipSoft repositories . . . 19

6. A team analysis: the mystery resonance 21
6.1. The data . . . 21
6.2. One script, ten tasks . . . 21
6.3. Who does what . . . 23
6.4. Round 0: the repository and its environment . . . 24
6.5. Round 1: your task, on your machine . . . 24
6.6. Round 2: the pull request and its checks . . . 24
6.7. Round 3: review, rebase, merge . . . 25
6.8. Round 4: watch the analysis grow . . . 25
6.9. Round 5: the answer . . . 25
6.10. Bonus tasks . . . 26

6.11. Setting it up . . . 26

II. The SHiP software chain **27**

7. Setting up the SHiP software **28**
7.1. What you need . . . 28
7.2. Where the software comes from . . . 28
7.3. The tutorial environment . . . 29
7.4. Versions used in this booklet . . . 29
7.5. Tutorial input files . . . 30

8. The big picture **31**
8.1. The guiding example . . . 31
8.2. The packages . . . 31
8.3. Phlex in one page . . . 32
8.4. Reproducibility in one paragraph . . . 33

9. The detector description **34**
9.1. GeoModel in a nutshell . . . 34
9.2. Coordinates, units and names . . . 34
9.3. What is implemented . . . 35
9.4. Building the geometry . . . 35
9.5. Looking at the geometry . . . 36

10. The geometry and field services **37**
10.1. The geometry service . . . 37
10.2. The field service . . . 38
10.2.1. The field-map format . . . 38
10.2.2. The command-line tools . . . 39

11. The event data model **41**
11.1. Plain structs, written as RNTuples . . . 41
11.2. The classes in this tutorial . . . 41
11.2.1. Generator particles . . . 42
11.2.2. Simulated particles and hits . . . 42
11.3. What the files look like . . . 43
11.4. Keeping old files readable . . . 44

12. The HNL sample **45**
12.1. EventCalc . . . 45
12.1.1. What comes out . . . 46
12.2. From EventCalc to the data model . . . 46

13. Simulation with aegir **48**
13.1. What aegir is . . . 48
13.2. The HNL workflow . . . 49
13.3. Sensitive volumes . . . 50
13.4. Running it . . . 51
13.5. Reproducibility and threads . . . 52
13.6. Going further . . . 52

14. Looking at events with sea-cucumber **53**
14.1. Three inputs and nothing else . . . 53
14.2. The REve display . . . 53
14.3. The web display . . . 54
14.4. View configuration . . . 54

15. Digitisation with Shannon **56**
15.1. From energy deposits to detector hits . . . 56
15.2. What the digitisers do today . . . 56
15.3. Running it . . . 57
15.4. A stream of hits, not a list of events . . . 58

16. Analysis from the files **60**
16.1. Reading the files . . . 60
16.2. Which file for which question . . . 61
16.3. The HNL analysis . . . 61
16.4. The veto . . . 62
16.5. Measuring momentum with the magnet . . . 62
16.6. Yields . . . 63
16.7. Going further . . . 63

17. Outlook: reconstruction with Trout **64**
17.1. Where it stands . . . 64
17.2. Questions for the discussion . . . 64

A. Command reference **66**

B. Data-model reference **69**

C. Troubleshooting **71**

D. Review scope and source revisions **73**
D.1. Checked primary sources . . . 73
D.2. Before the tutorial session . . . 73
D.3. Accessibility and validation . . . 74

How to use this booklet

This booklet accompanies the hands-on tutorial on the new SHiP software framework. It is built as a sequence of small steps (figure 1): every chapter says at its start what it needs and what you will be able to do afterwards, and ends with a short self-check. Each step uses only what the steps before it taught.

Part I is the way of working, with no SHiP software yet: git on your own, then with branches, then with other people and pull requests; pixi environments; continuous integration; and finally a small analysis that the whole group writes together, one reviewed and CI-checked pull request per person. Part II follows one physics example from beginning to end: a heavy neutral lepton (HNL, N) produced in the SHiP target that decays to $\pi^{\pm}\mu^{\mp}$ inside the decay volume. Every chapter of Part II adds one piece of the software chain, and each piece consumes what the previous one produced.

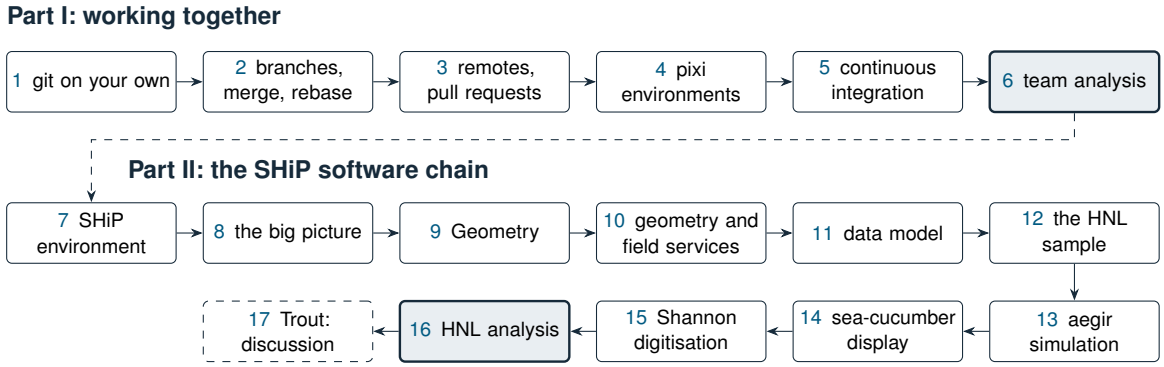


Figure 1.: The steps of the tutorial, numbered by chapter. Each step builds on the one before it; the shaded steps are the two places where everything learnt so far comes together. Part II starts once Part I is done.

Pitfall

This is a reviewed teaching draft, not an end-to-end validated release. The archive contains no HNL ROOT sample or EventCalc converter, field maps, or frozen common environment. The supplied workflow also selects passive material in some detectors. Its cut flow illustrates the interfaces; it is not a detector efficiency or sensitivity result. See appendix D for the checked source revisions and remaining prerequisites.

What you will learn

- how to work with git and GitHub the way the ShipSoft repositories do: commits, branches, merging versus rebasing, remotes, forks and pull requests with review (chapters 1 to 3);
- how to describe and reproduce a software environment with pixi: manifest, lock file, tasks and environments (chapter 4);
- how continuous integration checks every pull request, with GitHub Actions as in the ShipSoft repositories (chapter 5);
- how to build an analysis as a team, one reviewed, CI-checked pull request per person, by fitting a mystery resonance with PyROOT (chapter 6);

- how the SHiP detector is described in **GeoModel** (`ShipSoft/Geometry`) and how other packages reach it through the **geometry service** (`ShipSoft/geometry_service`);
- how magnetic field maps are stored and served (`ShipSoft/field_service`);
- what the event data model looks like on disk (`ShipSoft/data-model`);
- how to simulate events with **aegir** (`ShipSoft/aegir`), the Phlex-based simulation built on Geant4;
- how to look at them with the event display **sea-cucumber** (`ShipSoft/sea-cucumber`);
- how to digitise them with **Shannon** (`ShipSoft/Shannon`);
- how to analyse the resulting files directly from Python.

Reconstruction (**Trout**) is not part of the hands-on programme; it is the subject of the closing discussion (chapter 17).

Conventions

Commands to type in a terminal look like this:

```
pixi run phlex -c <(jsonnet hnl_sim.jsonnet)
```

Illustrative output is shown with a vertical rule on the left; exact log text may differ:

```
[info] sim_output_module: writing 'hnl_sim.root' ...
```

Package names are set in **bold sans serif**, files as `path/to/file`, and data-model fields as `pdgCode`. Lengths are in millimetres, momenta and energies in GeV, and times in nanoseconds throughout, matching the software. The coordinate system is the one of the SHiP geometry: the origin is the front face of the target, z points downstream along the beam, y points up (section 9.2).

The boxes have fixed meanings:

Exercise 0.1: Something for you to do

Exercises are numbered by chapter. Most take five to fifteen minutes.

Checkpoint

What you should see when the exercise worked. If you do not, ask a tutor before moving on: later chapters use the files you make here.

Note

Background, or where to read more.

Pitfall

A mistake that is easy to make, and how to recognise it.

The files you need

The runnable configurations and analysis scripts are supplied in the `framework-booklet/code/` directory of the accompanying archive. Inline API excerpts are explanatory. Extract the archive, then run:

```
cd framework-booklet
export CODE="$PWD/code"
export SHIP_TUTORIAL="$(dirname "$PWD")/ship-tutorial"
mkdir -p "$SHIP_TUTORIAL/work"
```

The commands in this booklet refer to that directory as `$CODE`; set both variables again in every new terminal. Use Bash for commands containing `<(...)`, and exit a package shell before activating another.

Getting help

Each package documents its classes in a Doxygen reference linked from its README, and the collaboration-wide documentation lives at `https://shipsoft.github.io/Documentation/`. Please report mistakes in this booklet to matclim@cern.ch.

Part I.

Working together

Chapter 1.

Git on your own

Where you are

Step: 1 of 6 in Part I: the first of three git chapters.

You need: a terminal, and `git` (`git --version` prints 2.23 or later). Nothing else: no GitHub account, no SHiP software.

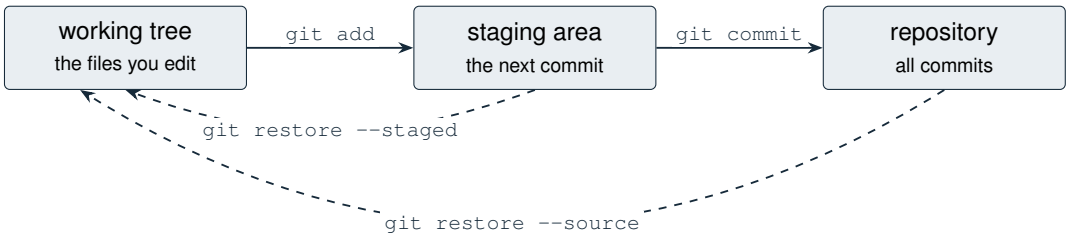
You will be able to: keep the history of your own work: make commits, look back at them, undo mistakes, and keep data out of the repository.

Every SHiP package lives in a git repository on GitHub, under `github.com/ShipSoft`, and every change to it arrives as a pull request. Part I builds up to that in small steps. This chapter is git on your own machine only; chapter 2 adds branches, chapter 3 other people and GitHub, chapters 4 and 5 the environment and the automatic checks around it, and chapter 6 puts everything together.

1.1. What git keeps

A git repository is a history of *snapshots* of a directory. Each snapshot is a *commit*: the complete state of every tracked file, an author, a date, a message, and a pointer to the commit it was made from (its *parent*). A commit is named by its hash, such as `c8dfe70`, which is computed from all of this, so a commit can never change: "changing" one means making a new commit with a new hash.

You work in three places:



`git add` chooses what goes into the next commit; `git commit` records it. Because you choose, one commit can hold one logical change even if you edited several things.

1.2. First steps

Exercise 1.1: Your first repository

Tell git who you are (once per machine), then make a repository:

```
git config --global user.name "Your Name"
git config --global user.email "your.name@cern.ch"
git config --global init.defaultBranch main
git config --global pull.rebase true
mkdir git-playground && cd git-playground
git init
echo "print('hello')" > hello.py
git status                          # hello.py is "untracked"
git add hello.py
git commit -m "feat: say hello"
git log --oneline
```

Note

Use the e-mail address of your GitHub account, so that GitHub can attach your commits to you later. `pull.rebase true` only matters from chapter 3 on; section 2.2 explains why SHiP wants it.

Exercise 1.2: Stage, look, undo

Change `hello.py` to print `'hello, SHiP'`.

1. `git status` and `git diff`: what does git say changed?
2. `git add hello.py`, then `git diff` and `git diff --staged`. Why is the first one empty now?
3. Unstage with `git restore --staged hello.py`, then throw the change away with `git restore hello.py`. Check with `cat hello.py`.

Pitfall

`git restore <file>` discards your uncommitted changes to that file for good: git never saw them, so it cannot give them back. What has been committed, on the other hand, is very hard to lose.

1.3. Good commits

One change per commit. A commit should do one thing that you can describe in one line: easier to review, to understand later, and to undo.

Commit messages. The SHiP repositories follow Conventional Commits: a type, an optional scope, and a short summary in the imperative.

`feat(task-07): fit with a Poisson likelihood`	a new feature
`fix(field): place the map at the magnet centre`	a bug fix
`docs: explain the sensitive-volume order`	documentation only
`refactor: test: style: chore:`	restructuring, tests, formatting, housekeeping

The release changelogs are generated from these messages, and in the ShipSoft repositories an automatic check rejects pull requests whose commits do not follow the convention (chapter 5).

Exercise 1.3: Read the history

Make two more commits in `git-playground`: change the message printed by `hello.py`, and add a second file. Then:

```
git log --oneline              # one line per commit, newest first
git show HEAD                  # the latest commit: message and change
git log -p hello.py            # every change ever made to one file
git diff HEAD~2 HEAD           # everything between two commits ago and
    ↪ now
```

`HEAD` is the commit you are on; `HEAD~1` its parent, `HEAD~2` its grandparent. Get an old version of a file back with `git restore --source HEAD~2 hello.py`, then commit it, or restore it again from `HEAD`.

1.4. What not to commit

Data, plots and build products do not belong in git: they are large, they change with every run, and they can be made again from the code. List them in a file called `.gitignore`, and git stops offering them to you.

Exercise 1.4: Ignore the outputs

```
touch data.root plot.pdf
git status                     # both are offered as "untracked"
printf '*.root\n*.pdf\n' > .gitignore
git status                     # now only .gitignore is
git add .gitignore && git commit -m "chore: ignore data and plots"
```

Before you move on, check that you can

- ☐ make a repository, stage exactly the changes you want, and commit them with a Conventional-Commit message;
- ☐ see what changed (`status`, `diff`, `diff --staged`) and read the history (`log`, `show`);
- ☐ undo an uncommitted change, and get an old version of a file back;
- ☐ keep data and outputs out of the repository with `.gitignore`.

Chapter 2.

Branches: merging and rebasing

Where you are

Step: 2 of 6 in Part I.

You need: chapter 1: your `git-playground` repository with a few commits.

You will be able to: develop on a branch, bring two lines of work together by merging or by rebasing, know which of the two SHiP uses and why, and resolve a conflict.

2.1. Branches

A *branch* is just a movable name for a commit. When you commit on a branch, the name moves forward to the new commit. `HEAD` is the branch you are on. Work on anything that is not finished happens on a branch of its own, so that `main` always works:

```
git switch -c feature          # create the branch "feature" and switch to it
# ... edit, add, commit ...
git switch main                # back to main; your files change accordingly
git branch                     # list branches; * marks the current one
```

Exercise 2.1: A branch of your own

In `git-playground`, create a branch `feature`, add a file `feature.py` and commit it. Switch back to `main`: where did `feature.py` go? Switch to `feature` again. Then look at both branches at once with `git log --oneline --graph --all`.

Meanwhile other work lands on `main`, and the two branches *diverge*. Sooner or later your branch has to be combined with `main`. There are two ways to do that.

2.2. Merging and rebasing

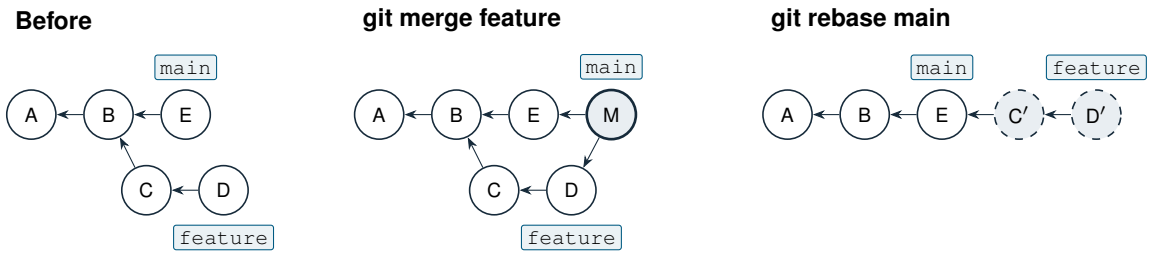


Figure 2.1.: Combining a branch with `main`. Arrows point from each commit to its parent. A merge adds a commit M with two parents. A rebase replays C and D on top of E as new commits C′ and D′ (dashed): same changes, new hashes, and a straight history; `main` can then simply move forward to D′ (a *fast-forward*).

Merging (`git merge feature`, run on `main`) keeps both lines of history exactly as they happened and joins them with a new *merge commit* that has two parents. Nothing is rewritten. The price is a history that branches and rejoins everywhere, which is hard to read and to bisect.

Rebasing (`git rebase main`, run on `feature`) takes the commits that are on `feature` but not on `main`, and re-applies them one by one on top of the current `main`. The result looks as if you had started your work from today's `main`: a straight line, with no merge commits. The price is that your commits are *replaced* by new ones with new hashes.

	merge	rebase
history	as it happened, with merge commits	straight line
your commits	kept, same hashes	rewritten, new hashes
conflicts	resolved once, in the merge commit	resolved per commit, as each is replayed
safe on shared branches	yes	**no**: others still have the old commits

What SHiP uses. The `main` branch of the ShipSoft repositories requires a *linear history*: merge commits are not accepted. Pull requests land by rebasing (or by squashing all their commits into one), through a merge queue (section 3.3). So when your branch falls behind `main`, you bring it up to date by *rebasing it*, not by merging `main` into it.

Pitfall

The golden rule: only rebase commits that nobody else has built on. Your own pull-request branch is fine; `main`, or a branch a colleague is also working on, is not. After rebasing a branch you had already pushed, push with `git push --force-with-lease` (chapter 3).

Exercise 2.2: Merge, then rebase

Continue from the previous exercise: switch to `main` and make one commit there, so the branches diverge. Copy the branch (`git branch feature-copy feature`). Merge `feature` into `main` and look at `git log --oneline --graph --all`. Undo the merge with `git reset --hard HEAD~1` (only ever on a playground), switch to `feature-copy`, run `git rebase main`, and look at the graph again. Compare the hashes of the rebased commits with the originals. Finally, `git switch main` and `git merge --ff-only feature-copy`: why does this one not create a merge commit?

2.3. Conflicts

When both sides changed the same lines, git cannot decide and stops. During a rebase it looks like this:

```
CONFLICT (content): Merge conflict in mystery_analysis.py
error: could not apply c8dfe70... feat(task-10): log-scale plot
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
```

and the file contains both versions between markers:

```
<<<<<<< HEAD
OUTPUT_PDF = "results/mystery_fit.pdf"
=======
OUTPUT_PDF = "mystery_fit_log.pdf"
>>>>>>> c8dfe70 (feat(task-10): log-scale plot)
```

Edit the file into what it should be, delete the three marker lines, then

```
git add mystery_analysis.py
git rebase --continue          # or: git rebase --abort, to go back to before
```

Pitfall

During a rebase the sides are the other way round from what you might expect: `HEAD` (the upper half) is the branch you are rebasing *onto*, i.e. the current `main`; the lower half is your own commit being replayed. Always run the tests again after resolving a conflict: a resolution that git accepts is not necessarily one that works.

Exercise 2.3: A conflict on purpose

```
printf 'OUTPUT = "plot.pdf"\n' > settings.py
git add settings.py && git commit -m "feat: settings"
git switch -c mine
printf 'OUTPUT = "mine.pdf"\n' > settings.py && git commit -am "feat: my
    ↪ name"
git switch main
printf 'OUTPUT = "theirs.pdf"\n' > settings.py && git commit -am "feat:
    ↪ their name"
git switch mine
git rebase main                # stops: CONFLICT
```

Open `settings.py`, decide what the line should be, remove the markers, `git add settings.py` and `git rebase --continue` (git opens an editor for the message: save and close it). Look at the graph. Then make the same conflict again and leave it with `git rebase --abort` instead.

Before you move on, check that you can

- ☐ create, switch between and list branches;
- ☐ explain, with a drawing, what a merge and a rebase do to the history;
- ☐ say why ShipSoft wants rebased, linear history, and which branches you may and may not rebase;
- ☐ resolve a conflict during a rebase, or abort it.

Chapter 3.

Remotes and pull requests

Where you are

Step: 3 of 6 in Part I.

You need: chapters 1 and 2. A GitHub account for the second half of the chapter, from section 3.2 on.

You will be able to: exchange commits with other copies of a repository (clone, fetch, pull, push), work with a fork, and take a change through a ShipSoft pull request.

3.1. Remotes

A *remote* is another copy of the repository that you exchange commits with. On GitHub it is usually called `origin`. Before going to GitHub, it helps to see that nothing magic happens there: a remote can just as well be a directory on your own disk.

Exercise 3.1: A server on your own machine

Play two people, Alice and Bob, sharing one "server" repository. Next to `git-playground` (not inside it):

```
git init --bare server.git              # a repository without a working
    ↪ tree
git clone server.git alice
git clone server.git bob                # (both warn: the repository is
    ↪ empty)
cd alice
echo "# Notes" > notes.md && git add notes.md
git commit -m "docs: start the notes"
git push -u origin main                 # Alice publishes main
cd ../bob
git pull origin main                    # Bob gets it
```

Now Bob starts work on a branch while Alice publishes more:

```
git switch -c bob-idea                  # in bob/
echo "an idea" > idea.md && git add idea.md && git commit -m "docs: add an
    ↪  idea"
cd ../alice
echo "more" >> notes.md && git commit -am "docs: more notes" && git push
cd ../bob
git fetch                               # download, change nothing
git log --oneline --graph --all         # origin/main is ahead of Bob's
    ↪ base
git rebase origin/main                  # replay Bob's idea on top of it
git push -u origin bob-idea
```

These are all the remote operations you need:

`git clone <address>`
: copies a repository and remembers where it came from as the remote `origin`.

`git fetch`
downloads new commits and updates the *remote-tracking* branches such as `origin/main`. It never touches your own branches or files, so it is always safe; afterwards you can look before you act (`git log main..origin/main`).

`git pull`
is `git fetch` followed by integrating the remote branch into your current branch: by rebasing, with the `pull.rebase` setting from section 1.2.

`git push -u origin <branch>`
uploads your branch. `-u` remembers the pairing, so later a plain `git push` suffices.

`git push --force-with-lease`
uploads a branch you have rebased, replacing your old commits on the remote. It refuses if somebody else pushed to that branch in the meantime; never use plain `--force`.

3.2. Forks

You cannot push to the ShipSoft repositories themselves. Like most open-source projects, SHiP uses *forks*: GitHub makes you a personal copy of the repository, you push your branches there, and you ask for them to be taken into the original with a pull request. So you work with two remotes.

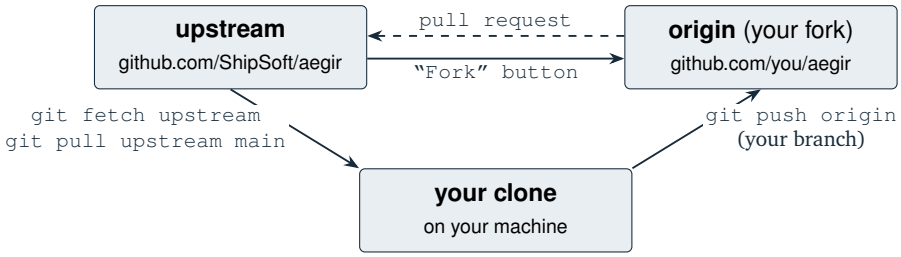


Figure 3.1.: The fork workflow. You read from `upstream`, write to your fork (`origin`), and ask for your branch to be taken into `upstream` with a pull request.

```
git clone git@github.com:<you>/aegir.git        # your fork becomes "origin"
cd aegir
git remote add upstream git@github.com:ShipSoft/aegir.git
git remote -v                                    # lists both
```

In section 3.1 there was one remote; with a fork, you fetch and pull from `upstream` and push to `origin`.

Note

GitHub accepts SSH (`git@github.com:...`) or HTTPS (`https://github.com/...`) addresses. SSH needs a key added to your GitHub account once; HTTPS needs a personal access token or the GitHub CLI (`gh auth login`) instead of a password. **[TBD: which one the tutorial machines are set up for]**

3.3. Pull requests the ShipSoft way

A pull request (PR) asks the maintainers of `upstream` to take the commits of one of your branches. The rules below are the ones configured for the ShipSoft repositories (in `ShipSoft/.github`,

`repo-config/`) and described in each package's `CONTRIBUTING.md`.

1. **Start from an up-to-date `main`**, on a new branch:

   ```
   git switch main && git pull upstream main
   git switch -c fix-field-translation
   ```

2. **Commit** in small, self-contained steps with Conventional-Commit messages.

3. **Push** the branch to your fork and **open the PR** against `main` of the ShipSoft repository, on the page GitHub shows you after the push, or with `gh pr create`. Say what the change does, why, and how you tested it.

4. **Checks.** Continuous integration builds and tests the branch automatically. The required checks are called "All checks passed" and, in most repositories, "Lint passed". A red check is yours to fix: push more commits to the same branch and the PR updates. Chapter 5 shows how this works.

5. **Review.** At least one approving review is required, and every review conversation must be resolved. A new push dismisses earlier approvals, so the reviewer sees what is actually merged.

6. **Up to date.** If `main` moved on in the meantime, the PR must be brought up to date, by rebasing, exactly as Bob did in section 3.1:

   ```
   git fetch upstream
   git rebase upstream/main
   # run the tests again
   git push --force-with-lease
   ```

7. **Merge.** An approved, green PR goes into the merge queue, which rebases it onto `main`, runs the checks once more, and only then lands it. Merge commits are not possible, and nobody, maintainers included, can force-push to or delete `main`.

8. **Afterwards** update your own copies and delete the branch:

   ```
   git switch main && git pull upstream main && git push origin main
   git branch -d fix-field-translation
   ```

You will take a real pull request through all of these steps in chapter 6. Two things in step 4 still need explaining first: the environment the checks run in (chapter 4), and the checks themselves (chapter 5).

3.4. Git cheat sheet

Command	What it does
`git status`	what changed, what is staged, which branch
`git diff [--staged]`	changes not yet staged [already staged]
`git add <file>`	stage a change for the next commit
`git commit -m "type: summary"`	record the staged changes
`git log --oneline --graph --all`	the history as a graph
`git restore <file>`	discard unstaged changes to a file
`git switch -c <branch>`	create a branch and switch to it
`git switch <branch>`	switch to an existing branch
`git fetch upstream`	download, change nothing locally
`git pull upstream main`	fetch and rebase the current branch onto it
`git rebase upstream/main`	replay your branch on the latest main
`git rebase --continue / --abort`	after resolving a conflict / give up
`git push -u origin <branch>`	publish a branch to your fork
`git push --force-with-lease`	publish a rebased branch (yours only!)

Before you move on, check that you can

- ☐ clone, fetch, pull and push, and say what each one changes;
- ☐ bring an out-of-date branch up to date with a rebase and push it safely;
- ☐ set up a fork with `origin` and `upstream`;
- ☐ list the steps a ShipSoft pull request goes through before it is merged.

Chapter 4.

Reproducible environments with pixi

Where you are

Step: 4 of 6 in Part I.

You need: chapter 1: you can commit. A network connection to download packages.

You will be able to: create a project environment from a manifest and a lock file, run commands and tasks in it, add packages and a second environment, and recognise how the ShipSoft repositories use all of this.

4.1. Why an environment, and why pixi

"It works on my machine" is the oldest problem in shared software. An analysis depends on Python, ROOT, and everything ROOT depends on; the SHiP packages add Geant4, GeoModel, Phlex and compilers. Two people with different versions of any of these can get different results, or none.

pixi solves this per project. It installs packages from conda channels (`conda-forge` for the general HEP stack, `prefix.dev/ship` for the SHiP packages) into a directory inside the project, `.pixi/`. Nothing is installed system-wide, nothing needs administrator rights, and every project has exactly the versions it asks for. Every ShipSoft repository is a pixi project, and so is the team repository of chapter 6.

A pixi project is described by two files, and both are committed to git:

	`pixi.toml`, the *manifest*	`pixi.lock`, the *lock file*
written by	you	pixi, never by hand
says	what the project needs: `root_base >=6.36`	exactly what was installed: every package, version and build, per platform
changes	when you change the requirements	when pixi solves again (`pixi add`, `pixi update`)
gives	the recipe	the same environment for everybody, and for CI

4.2. Installing pixi

```
curl -fsSL https://pixi.sh/install.sh | bash
# open a new terminal, then:
pixi --version
```

Pitfall

pixi keeps downloaded packages in a cache (`pixi info` shows where) and the environments in each project's `.pixi/`. Both can take several gigabytes. On machines with a small home quota, such as lxplus, point the cache at a larger disk with `export PIXI_CACHE_DIR=<somewhere big>` in your shell's start-up file. **[TBD: recommended location on the tutorial machines]**

4.3. Your first workspace

Exercise 4.1: An environment with PyROOT

Next to your git playground:

```
mkdir pixi-playground && cd pixi-playground
git init
pixi init                                    # writes pixi.toml, .gitignore,
    ↪ .gitattributes
pixi add python "root_base>=6.36"            # solves, writes pixi.lock,
    ↪ installs
pixi run python -c "import ROOT; print(ROOT.gROOT.GetVersion())"
```

Open `pixi.toml`: what did `pixi add` write into it? Look at the size of `pixi.lock` and search it for `root_base`: which exact version and build did you get? What does the new `.gitignore` keep out of git? Commit the project:

```
git add pixi.toml pixi.lock .gitignore .gitattributes
git commit -m "feat: environment with PyROOT"
```

Checkpoint

A ROOT version of 6.36 or later printed, and one commit containing the four files, but not `.pixi/`.

There are two ways to use the environment:

`pixi run <command>`
: runs one command inside the environment and returns. Use it in scripts, in CI, and whenever you can.

`pixi shell`
: opens a shell with the environment active until you `exit`. Convenient for interactive work; leave it before you enter the shell of another project, or you get a mixture of both.

4.4. Tasks

Typing the same long commands is error-prone, and the next person will type them differently. A *task* gives a command a name, stored in the manifest, so that everybody, including CI, runs exactly the same thing.

Exercise 4.2: Name your commands

```
echo 'import ROOT; print("hello from ROOT", ROOT.gROOT.GetVersion())' >
    ↪ hello.py
pixi task add hello "python hello.py"
pixi run hello
pixi task list
```

Look at the `[tasks]` table in `pixi.toml`. Tasks can depend on other tasks and take arguments; edit the table by hand to:

```
[tasks]
hello = "python hello.py"
greet = { cmd = "echo Hello, {{ name }}", args = ["name"] }
all = { cmd = "echo done", depends-on = ["hello"] }
```

and try `pixi run greet SHiP` and `pixi run all`. Commit.

4.5. More than one environment

Some tools are needed only for some jobs: a linter, a benchmark tool, a documentation generator. pixi groups them into *features* and combines features into named *environments*, so that the linter does not end up in the analysis environment.

Exercise 4.3: A separate environment for the linter

```
pixi add --feature lint ruff
pixi workspace environment add lint --feature lint --no-default-feature
pixi task add --feature lint lint "ruff check ."
pixi run lint                  # pixi picks the environment that has the
    ↪ task
pixi run -e lint ruff --version
```

Read the `[feature.lint]` and `[environments]` tables that appeared in `pixi.toml`. Write an unused `import os` into `hello.py` and run `pixi run lint` again. Remove it, and commit.

4.6. Keeping everybody on the same versions

`pixi install --locked`
: installs exactly what `pixi.lock` says, and fails if the lock no longer matches the manifest. This is what CI runs: a change of requirements must come with its lock file.

`pixi update`
: solves again and moves to newer versions within the manifest's ranges. The new `pixi.lock` is a change like any other: commit it, on its own, so that its effect can be tested and reviewed. In the ShipSoft repositories a bot opens such a pull request every month (chapter 5).

`pixi list`
: shows the installed packages and versions.

Pitfall

Never edit `pixi.lock` by hand, and do not resolve conflicts inside it (the `.gitattributes` that `pixi init` writes tells git not to try). If a rebase conflicts on `pixi.lock`, take the version of `main` (`git checkout upstream/main -- pixi.lock`), run `pixi install` to solve again with your manifest, and `git add` the result. Better still: do not change the environment in a pull request that is about something else.

4.7. How the ShipSoft repositories use pixi

Every repository has a `pixi.toml` and a `pixi.lock` at its top. Their channels are the SHiP channel on `prefix.dev` and `conda-forge`. Building and testing are tasks, chained with `depends-on`, so that one command does everything that is needed. From `ShipSoft/Shannon`:

```
build = { cmd = "cmake --build build -j8", depends-on = ["configure"] }
install = { cmd = "cmake --install build", depends-on = ["build"] }
test = { cmd = "ctest --test-dir build --output-on-failure", depends-on = ["
    ↪ build"] }
smoke = { cmd = "scripts/smoke.sh", depends-on = ["build"] }
```

so `pixi run test` configures, builds and tests a fresh checkout.

- **A `lint` environment** holds the formatting and lint tools (`prek`, `ruff`, `clang-format`, `codespell`, `commitizen`, ...). `pixi run lint` checks every file; `pixi run install-hooks` installs git hooks that run the same checks at every commit.
- **Other environments** for special jobs, such as `bench` for benchmarks in **aegir**.
- **A commented-out `dev` feature** builds a dependency from a local checkout, for changing two packages at once (see the developer guide of the SHiP documentation). It stays commented out in commits, together with the lock-file changes it causes.
- **CI runs the same tasks**, with `--locked` (chapter 5). If `pixi run --locked test` passes on your machine, the build check passes in CI.

Before you move on, check that you can

- ☐ say what belongs in `pixi.toml` and what in `pixi.lock`, and why both are committed;
- ☐ create a workspace, add packages, and run a command in it with `pixi run` or `pixi shell`;
- ☐ define tasks, and a second environment from a feature;
- ☐ reproduce an environment exactly with `--locked`, and update it on purpose.

Chapter 5.

Automatic checks: continuous integration

Where you are

Step: 5 of 6 in Part I.

You need: chapter 3 (pushing to GitHub) and chapter 4 (tasks and `--locked`); your `pixi-playground`; a GitHub account.

You will be able to: read a GitHub Actions workflow, set one up, find out why a check failed and reproduce it locally, and recognise the checks every ShipSoft pull request must pass.

5.1. What CI does

Continuous integration (CI) means: every time somebody pushes, a fresh machine checks out the commit, builds it, and runs the tests and the linters. The result appears on the pull request as a *check*, green or red. Nobody has to remember to run the tests, the reviewer does not have to run them either, and they run in a clean environment rather than on somebody's carefully configured laptop. A repository can make checks *required*: a pull request with a red required check cannot be merged.

CI only works if it runs exactly what you run. That is what the pixi tasks and the lock file of chapter 4 are for: CI runs `pixi run --locked <task>`, the same command, in the same environment, that you run at home.

5.2. GitHub Actions in five words

GitHub's CI is called GitHub Actions. Its vocabulary:

workflow
: a YAML file in `.github/workflows/` in the repository.

events
: what starts it, under `on:`: a push, a pull request, the merge queue, a schedule, a button.

jobs what it does. Each job runs on a fresh virtual machine (`runs-on: ubuntu-latest`); jobs run in parallel unless one `needs` another.

steps
: the commands of a job, in order. A step either `uses` a ready-made *action* (check out the code, install pixi) or `run`s shell commands.

logs everything a step prints, kept on GitHub, one click away from the pull request.

5.3. Your first workflow

code/ci/playground-ci.yml

```yaml
# SPDX-FileCopyrightText: 2026 CERN for the benefit of the SHiP Collaboration
# SPDX-License-Identifier: LGPL-3.0-or-later
#
# A first CI workflow for your pixi playground (booklet chapter on CI).
# Copy it to .github/workflows/ci.yml in that repository, commit, and push.
name: CI

on:
  push:
  pull_request:
  workflow_dispatch:

jobs:
  hello:
    name: Hello from CI
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: prefix-dev/setup-pixi@v0.11.0
        with:
          locked: true
      - name: Run the same task you run at home
        run: pixi run --locked hello
```

Three steps: `actions/checkout` fetches the commit being tested; `prefix-dev/setup-pixi` installs pixi and the environment, from `pixi.lock` (`locked: true`); and the last step runs your task.

Exercise 5.1: CI for your playground

On GitHub, create a new, empty repository `pixi-playground` (no README, no licence). Then, in your `pixi-playground`:

```
mkdir -p .github/workflows
cp $CODE/ci/playground-ci.yml .github/workflows/ci.yml
git add .github && git commit -m "ci: run hello on every push"
git remote add origin git@github.com:<you>/pixi-playground.git
git push -u origin main
```

Open the repository's *Actions* tab and follow the run. Click on the job and open each step: where does it print the ROOT version?

Pitfall

CI runs on Linux. `pixi init` on a Mac writes only your Mac's platform into `pixi.toml`, and `pixi run --locked` then fails in CI because the lock file has nothing for `linux-64`. Add it with `pixi workspace platform add linux-64` and commit the new lock file.

Exercise 5.2: Red, then green

Break the task on a branch and propose the breakage to yourself:

```
git switch -c broken
echo 'raise SystemExit("deliberately broken")' >> hello.py
git commit -am "test: break hello on purpose"
git push -u origin broken
```

Open a pull request from `broken` into `main` of your own repository. Watch the check turn red, open its log, and find the error. Reproduce it locally with the very same command, `pixi run --locked hello`. Then repair `hello.py`, commit, `git push`, and watch the same pull request turn green. Close it without merging.

5.4. The workflow of the team repository

The team repository of chapter 6 has a workflow laid out like the ShipSoft ones. Read it once now; you will see it run on your pull request.

code/teamwork/template/.github/workflows/ci.yml

```yaml
# SPDX-FileCopyrightText: 2026 CERN for the benefit of the SHiP Collaboration
# SPDX-License-Identifier: LGPL-3.0-or-later
#
# Continuous integration of the team analysis, laid out like the ShipSoft
# repositories: one job per kind of check, and a final job whose name is the
# single required status check of the repository ("All checks passed").

name: CI

on:
  pull_request:           # every push to a pull request
    branches: [main]
  push:                   # every new commit on main
    branches: [main]
  merge_group:            # the merge queue re-checks before it merges
  workflow_dispatch:      # a "Run workflow" button in the Actions tab

concurrency:              # a new push cancels the run for the previous one
  group: ci-${{ github.event.pull_request.number || github.ref }}
  cancel-in-progress: true

jobs:
  tasks:
    name: Task checks
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7               # the commit being tested
      - uses: prefix-dev/setup-pixi@v0.11.0     # pixi, and the environment from pixi.
    ↪ lock
        with:
          locked: true
      - name: Test every task
        run: pixi run --locked check

  lint:
    name: Lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
```

```
      - uses: prefix-dev/setup-pixi@v0.11.0
        with:
          environments: lint
          locked: true
      - name: ruff
        run: pixi run --locked -e lint lint

  all-checks:
    name: All checks passed
    if: always()
    runs-on: ubuntu-latest
    needs: [tasks, lint]
    steps:
      - name: Check status of all jobs
        run: |
          if [[ "${{ contains(needs.*.result, 'failure') || contains(needs.*.result,
  ↪ 'cancelled') }}" == "true" ]]; then
            echo "One or more jobs failed or were cancelled"
            exit 1
          fi
```

Events.
Pull requests and pushes to `main`, as before, plus `merge_group`: the merge queue runs the checks once more on the rebased commit before it lands (section 3.3).

Concurrency.
A new push to a pull request cancels the run for the previous push, which nobody needs any more.

Two jobs
run in parallel: `tasks` runs `pixi run --locked check` (all ten task checks of chapter 6), and `lint` runs `ruff` in its own pixi environment.

One summary job,
"All checks passed", needs both and fails if either failed or was cancelled. Its name is the one required check of the repository. Jobs can be added or renamed later without touching the repository settings; `if: always()` makes it report a failure instead of being silently skipped.

Exercise 5.3: Predict the checks

Without running anything, answer from the workflow above:

1. A pull request adds a correct task 4 but also an unused `import`. Which jobs are green, which red, and can the pull request be merged?
2. You push twice within a minute. How many runs complete?
3. Which single command reproduces the `tasks` job on your machine, and which the `lint` job?

5.5. CI in the ShipSoft repositories

The ShipSoft repositories share their CI through *reusable workflows* in `ShipSoft/.github`: each repository's workflow is a few lines that call a common one. From `ShipSoft/Shannon`'s `.github/workflows/pixi-build.yml`:

```
jobs:
  build:
    uses: ShipSoft/.github/.github/workflows/pixi-cmake-build.yml@main
    with:
      tasks: '["configure", "build", "install", "test", "smoke"]'
```

followed by the same "All checks passed" summary job as in the team repository. What runs, and what it means for you:

Workflow	What it checks
`pixi-cmake-build`	runs the listed pixi tasks with `--locked`, compiling through a cache ("All checks passed")
`prek`	the lint hooks of the `lint` environment on all files, and posts their output on the pull request ("Lint passed")
`commit-check`	every commit message of the pull request against Conventional Commits (part of "Lint passed")
`clang-tidy`	C++ guidelines, reported only on the lines your pull request changes
`pixi-lock-update`	once a month, opens a pull request with an updated `pixi.lock`
`physics-metrics`	physics distributions compared with a reference (`ShipSoft/ship-ci-metrics`)

Note

Run the lint checks before you push and the "Lint passed" check is a formality: `pixi run lint` checks everything, and `pixi run install-hooks` makes the same checks run at every `git commit`. When a check is red anyway, the log of the failed step names the pixi task; rerun exactly that task locally.

Exercise 5.4: Optional: read a real run

Open `https://github.com/ShipSoft/aegir/actions`, choose a recent run of "Pixi Build", and find the log group of `pixi run --locked test`. How long did the build take, and how many tests ran?

Before you move on, check that you can

- ☐ name the parts of a workflow: events, jobs, steps;
- ☐ set up CI for a pixi project and watch it run;
- ☐ go from a red check to its log, and reproduce the failure with the same pixi task locally;
- ☐ explain why a summary job, rather than every job, is the required check, and what `merge_group` is for.

Chapter 6.

A team analysis: the mystery resonance

Where you are

Step: 6 of 6 in Part I: everything together.

You need: chapters 1 to 5; a GitHub account; some Python. Still no SHiP software: the team repository brings its own pixi environment with PyROOT.

You will be able to: take one piece of a shared analysis from an empty function to a merged pull request, through local tests, CI and review, while the team's analysis grows with every merge.

The whole group writes one small analysis together. Each person writes one piece, in a pull request that CI checks and a teammate reviews, and the analysis only works when everybody's pieces fit. The exercise runs in rounds; each round uses one of the earlier chapters.

Round	What you do	Uses
0	fork, clone, install the environment	chapters 3 and 4
1	write and test your task on your machine	chapters 1 and 4
2	open the pull request, get the checks green	chapters 3 and 5
3	review, rebase, merge	chapters 2 and 3
4	watch the analysis grow	chapters 3 and 4
5	find the particle	PyROOT

6.1. The data

The file `mystery.root` holds 20 000 measurements of the invariant mass of the decay products of an unknown particle, in GeV. They are stored in an RNTuple called `events`, with a single field, `mass`, of type `float`. The line shape is a relativistic resonance, described by a Breit–Wigner distribution:

$$f(m) \;=\; \frac{1}{\pi}\,\frac{\Gamma/2}{(m-M)^2+\Gamma^2/4},$$

with mass M and width Γ (the full width at half maximum). Your team has to measure M and Γ with their uncertainties, make a plot of the fit, and find out which particle this is. Nobody is told the answer.

6.2. One script, ten tasks

The analysis lives in one file, `mystery_analysis.py`, in a repository of its own, the *team repository* (section 6.11). It is cut into ten tasks, each one a Python function that is empty to start with:

	Function	Input → Output
1	`open_file`	file name → RNTuple reader
2	`read_masses`	reader → list of masses
3	`summarise`	masses → count, mean, median, central 68% interval
4	`book_histogram`	summary → empty histogram `h_mystery`
5	`fill_histogram`	histogram, masses → filled histogram
6	`make_model`	histogram → Breit–Wigner `f_func` with starting values
7	`fit_model`	histogram, model → fit result
8	`extract_results`	fitted model → mass, width, errors, fit quality
9	`identify_particle`	results → best-matching known particle
10	`save_plot`	histogram, model, results → `mystery_fit.pdf`

Above each function, a comment gives the contract: what the function receives (*Input*), what it must return (*Output*), the ROOT calls that are useful (*Relevant*), some *Hints*, and how to test it. It tells you what to use, not what to write. Task 7, for example:

code/teamwork/template/mystery_analysis.py

```
# ============================================================================
# TASK 7 -- Fit                                                owner: _____________
# ----------------------------------------------------------------------------
# Input:    h_mystery    (ROOT.TH1F) filled histogram from task 5
#           f_func       (ROOT.TF1) model with starting values from task 6
# Output:   fit_result   (the object returned by h.Fit(..., "S")); after the
#                        call, f_func holds the fitted parameters
# Relevant: h.Fit("gaus")              fits a built-in Gaussian
#           h.Fit(f_func, options)     fits your own TF1; useful options:
#             "S" return the fit result   "L" Poisson likelihood (counts!)
#             "R" use the TF1's range     "Q" quiet   "0" do not draw
#           fit_result.Status()        0 means the fit converged
#           fit_result.Print("V")      parameters, errors, correlations
# Hints:    Which option is right for a histogram of counts, chi-square or
#           likelihood? Try both and compare. If Status() is not 0, raise a
#           RuntimeError: a failed fit must not produce a number.
# Check:    pixi run check-task 7
# ============================================================================
def fit_model(h_mystery, f_func):
    # ---- TASK 7: your code below (delete the "raise" line) ----------------
    raise NotImplementedError("task 7: fit_model")
    # ---- TASK 7: your code above ------------------------------------------
```

The contract is what makes the pieces fit: the owner of task 8 relies on task 7 returning exactly what its Output line promises. Do not rename your function or change its arguments, and keep your code between the two marker lines.

Running the chain

At the bottom of the file, a function written by the organisers calls the ten tasks in order. It stops, without an error, at the first task that is not written yet:

```
$ pixi run analysis
Pixi task (analysis): python mystery_analysis.py
--- task  1/10: open_file
mystery.root: 20000 entries in 'events'
--- task  2/10: read_masses
--- task  3/10: summarise
  n = 2e+04, min = 20.03, max = 199.7, mean = ..., median = ...
```

```
--- task  4/10: book_histogram
--- task  5/10: fill_histogram
--- task  6/10: make_model is not written yet; the chain stops here (5 of 10 tasks
    ↪ done)
```

Every merged pull request lets the chain run one step further, until it ends with `all 10 tasks done`.

Testing your task on its own

You do not have to wait for your teammates. `check_task.py` tests one task with inputs it makes up itself, from a toy Breit–Wigner sample whose answer it knows ($M = 50$ GeV, $\Gamma = 4$ GeV: *not* the mystery particle), and checks your function's Output against its contract:

```
$ pixi run check-task 4
Pixi task (check-task): python check_task.py 4
task  4: FAILED
      the histogram must be named 'h_mystery', got 'h'
$ pixi run check-task 4
Pixi task (check-task): python check_task.py 4
task  4: passed
```

The same checker, for all ten tasks at once, is what CI runs on every pull request (section 5.4).

6.3. Who does what

With ten participants, everyone takes one task. With fewer, some take two or three neighbouring tasks; with more, pairs share a task, or the extra people take the bonus tasks of section 6.10.

People	Tasks per person
10	1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10
9	1 | 2 | 3 | 4 | 5 | 6 | 7 | 8+9 | 10
8	1 | 2+3 | 4 | 5 | 6 | 7 | 8+9 | 10
7	1 | 2+3 | 4+5 | 6 | 7 | 8+9 | 10
6	1+2+3 | 4+5 | 6 | 7 | 8+9 | 10
5	1+2+3 | 4+5 | 6 | 7+8 | 9+10

Your reviewer is the owner of the next task, the person who will use what your function returns (task 10 is reviewed by the owner of task 1). Write your name and your reviewer's next to your task in the table of the repository's `README.md`, in your first commit.

6.4. Round 0: the repository and its environment

Fork the team repository on GitHub (“Fork” button), then:

```
git clone git@github.com:<you>/<team-repo>.git
cd <team-repo>
git remote add upstream git@github.com:<organisers>/<team-repo>.git
pixi install                                  # Python and ROOT, from pixi.lock
cp <wherever-it-is>/mystery.root .            # data: not under version control
pixi run analysis                             # stops at task 1 for now
pixi run check                                # 10 not written yet, 0 failed
```

`pixi task list` shows what else the repository offers.

Checkpoint

`pixi run analysis` stops at task 1 without an error, and `pixi run check` reports `0 passed, 10 not written yet, 0 failed`.

6.5. Round 1: your task, on your machine

One branch, one task:

```
git switch -c task-04-book-histogram        # task-NN-short-description
# ... write your function, and your name in README.md ...
pixi run check-task 4                       # until it says "passed"
pixi run lint                               # until ruff is happy
git add mystery_analysis.py README.md
git commit -m "feat(task-04): book the mystery histogram"
```

Commit as often as you like on your branch: small steps are easier to review and to undo.

6.6. Round 2: the pull request and its checks

```
git push -u origin task-04-book-histogram
```

Open the pull request against `main` of the team repository (the page GitHub shows after the push has a button for it). Describe what you did, paste the output of `pixi run check-task 4`, and ask your reviewer for a review.

CI starts by itself (section 5.4). Wait for “All checks passed”. If it is red, open the log of the failing job, reproduce the failure with the same pixi task on your machine, fix it, commit, and `git push`: the pull request updates and CI runs again.

6.7. Round 3: review, rebase, merge

As a reviewer. Read the change in the "Files changed" tab. Does it do what the contract says, and nothing else? Would you, as the next task's owner, be happy to receive its output? Comment on lines you do not understand. Approve only when you are satisfied and the checks are green.

As an author. Answer every comment, change what needs changing, and push again; a new push dismisses earlier approvals, so ask for a new one.

When your pull request is out of date. Every merge moves `main` forward, and GitHub then asks for your branch to be brought up to date. Rebase:

```
git fetch upstream
git rebase upstream/main
pixi run check-task 4
git push --force-with-lease
```

Since every task lives in its own function, this rebase almost never conflicts. If it does, section 2.2 explains what to do. An approved, green, up-to-date pull request is merged by the organisers (or by the merge queue, if the repository has one).

6.8. Round 4: watch the analysis grow

After every merge, everybody can run the team's analysis as far as it goes:

```
git switch main && git pull upstream main
pixi run analysis
git switch task-04-book-histogram           # back to your own work
```

6.9. Round 5: the answer

Exercise 6.1: Find the mystery particle

When the chain reaches `all 10 tasks done`, open `mystery_fit.pdf`.

1. What are the mass and the width, with their uncertainties?
2. Which particle is it? Is the fitted width compatible with that particle's, too, or only the mass?
3. Is the fit good? Look at χ^2/ndf and at the tails on a logarithmic scale.
4. Change one thing at a time (the binning in task 4, chi-square versus likelihood in task 7, the starting values in task 6) and rerun. Which changes move the result by more than its uncertainty?

6.10. Bonus tasks

For those who finish early, or for an eleventh and twelfth participant. Each is a separate pull request.

B1: pulls
Under the plot, add a second pad with $(n_i - f(m_i))/\sigma_i$ per bin. Does the model describe the tails?

B2: chi-square versus likelihood
Fit with and without the `"L"` option and tabulate both results. Which is biased, and why, for bins with few entries?

B3: a background
The organisers have a second sample, `mystery_bkg.root`, with a falling background under the peak. Add an exponential to the model (`"[0]*TMath::BreitWigner(x,[1],[2]) + [3]*exp(-[4]*x)"`) and compare the result with and without it.

B4: detector resolution
Real detectors smear the mass. The sample `mystery_res.root` has a Gaussian resolution added; fit it with a Voigt profile (`TMath::Voigt`), the convolution of a Breit–Wigner with a Gaussian. What happens to the width if you ignore the resolution?

6.11. Setting it up

The team repository is **[TBD: URL of the team repository]**. It starts from the files in `code/teamwork/template/`: the skeleton `mystery_analysis.py`, the checker `check_task.py`, the pixi manifest `pixi.toml` with its lock file, the lint settings `ruff.toml`, the CI workflow `.github/workflows/ci.yml`, a `README.md` with the task table, the rules in `CONTRIBUTING.md`, and a `.gitignore` that keeps data, plots and `.pixi/` out of the history.

Part II.

The SHiP software chain

Chapter 7.

Setting up the SHiP software

Where you are

Step: 1 of 11 in Part II.

You need: Part I, in particular chapter 4; a Linux x86_64 machine with enough disk space.

You will be able to: install the SHiP software chain in one pixi environment, check that it works, and have the tutorial input files at hand.

7.1. What you need

- A Linux x86_64 machine: the SHiP packages are built for `linux-64` only. **[TBD: lxplus / institute machines / provided VMs; macOS users?]**
- About **[TBD: N]** GB of free disk space for the environment and the tutorial files.
- pixi (chapter 4), the package manager used by the new-stack repositories. It installs everything else, compilers included, without root access.
- A web browser, for the **sea-cucumber** web display (chapter 14).

7.2. Where the software comes from

Released components of the new stack are distributed as conda packages on the `prefix.dev/ship` channel; other dependencies are resolved from the SHiP channel and `conda-forge`. The recipes are in `ShipSoft/ship-conda-recipes`. Each repository carries a `pixi.toml` that pins its dependencies and a `pixi.lock` that freezes the exact versions, so the same committed lock file can reproduce a resolved environment. Use the tutor-provided revision and lock file; cloning moving branches does not freeze the software. `pixi install --locked` prevents silent re-resolution.

Note

As in chapter 4, two commands do almost everything:

`pixi run <task or command>`
runs one command inside the environment, e.g. `pixi run build` or `pixi run phlex -c ...`.

`pixi shell`
opens a shell with the environment active until you `exit`. This is the convenient mode for this tutorial.

7.3. The tutorial environment

[TBD: Final recipe. Either a single tutorial `pixi.toml` pulling in all packages at the versions in table 7.1, or the per-repository checkouts below.]

If you work with the individual repositories, check them out side by side; the SHiP developer guide calls this the *sibling layout*, and some configuration files refer to each other by relative path:

```
cd "$SHIP_TUTORIAL"
for r in Geometry aegir Shannon sea-cucumber field_service; do
  git clone https://github.com/ShipSoft/$r.git
done
(cd Geometry && pixi run geometry)      # builds the detector description
(cd aegir    && pixi run build)         # builds the simulation plugins
(cd Shannon  && pixi run build)         # builds the digitisation plugins
# Before building sea-cucumber, pin shipdatamodel to >=0.5,<0.6
# in its pixi.toml and freeze the resolved environment with the tutors.
(cd sea-cucumber && pixi run build)     # after that compatibility check
```

Inside **aegir** and **Shannon**, the pixi activation script (`activate.sh`) sets the environment variables the framework needs:

Variable	Meaning
`PHLEX_PLUGIN_PATH`	where `phlex` looks for plugins: the package's `build/` directory, then the environment's `lib/`
`SHIPGEOMETRY_ROOT`	a bare geometry file name such as `ship_geometry.db` is looked up in the current directory, then in `$SHIPGEOMETRY_ROOT/share/geometry/`
`SHIPFIELD_ROOT`	a bare field-map file name is looked up in `$SHIPFIELD_ROOT/share/field/`

7.4. Versions used in this booklet

Package	conda package	Version
Geometry	`shipgeometry`	**[TBD: x.y]**
geometry service	`shipgeometryservice`	**[TBD: x.y]**
field service	`field-service`	**[TBD: x.y]**
data model	`shipdatamodel`	0.5.x
ship-random	`ship-random`	**[TBD: x.y]**
aegir	`aegir`	**[TBD: x.y]**
Shannon	`shannon`	**[TBD: x.y]**
sea-cucumber	(built from source)	**[TBD: commit]**
Phlex	`phlex`	$\geq$ 0.4.1

Table 7.1.: Target version constraints and values still to freeze. This is not a tested environment manifest.

Pitfall

The data model is at the centre of a version chain: **aegir** writes it, **Shannon** and **sea-cucumber** read it, and your analysis reads it too. `shipdatamodel` 0.6 renamed camelCase data-model members to snake_case, for example `pdgCode` to `pdg_code`. This booklet uses the 0.5 names, which is what **aegir** and **Shannon** are built against at the time of writing. Appendix B lists both spellings. If a script fails with an unknown member, check which data-model version your environment has before anything else.

Pitfall

As checked on 15 October 2026, **field service**'s development manifest requires `shipdatamodel >=0.6.0`, while **aegir** requires `>=0.5,<0.6`. These constraints cannot be satisfied in one environment. Use a compatible released field-service build with the simulation, or keep field-map tools in a separate process and verify their file format against that build. Do not add the current field-service checkout to the simulation environment without resolving this conflict. **sea-cucumber**'s unbounded data-model dependency also needs a compatible pin before a fresh solve.

7.5. Tutorial input files

Two kinds of input are provided rather than generated during the tutorial:

- **HNL samples**: events from **EventCalc**, already converted to the SHiP data model (chapter 12). Location: **[TBD: path or URL]**.
- **Field maps**: the muon-shield and spectrometer maps in the SHiP field-map format (section 10.2). Location: **[TBD: path or URL]**.

Copy them to `$SHIP_TUTORIAL/work/`. Keep the sample metadata (mass, lifetime, mixing, generator revision, coordinate transform, weights and exposure) beside the files.

Exercise 7.1: Check your environment

From the `aegir` directory, open a shell in the environment and check that the tools respond:

```
pixi shell
which phlex jsonnet
# In the Geometry environment, check gmex separately: which gmex
echo $PHLEX_PLUGIN_PATH
python -c "import ROOT; print(ROOT.gROOT.GetVersion())"
```

Checkpoint

Paths for Phlex and Jsonnet inside the pixi environment, a plugin path that starts with the `aegir/build` directory, and a ROOT version satisfying the frozen environment (aegir currently requires 6.40 or later). Geometry viewing additionally needs `gmex` and a graphical session; do not assume it is installed in the aegir environment.

Chapter 8.

The big picture

Where you are

Step: 2 of 11 in Part II.

You need: chapter 7: a working environment.

You will be able to: name the packages of the chain and the file each one produces, read a Phlex workflow, and explain why runs are reproducible.

8.1. The guiding example

A heavy neutral lepton mixes with the active neutrinos. At SHiP it is produced mostly in decays of charm and beauty mesons in the target, flies through the muon shield without interacting, and decays somewhere in the 50 m long decay volume. We follow its decay

$$N \to \pi^{\pm}\mu^{\mp},$$

a useful benchmark signature: two charged tracks from a common vertex inside the decay volume, no activity in the veto taggers, and a reconstructed invariant mass distributed around the HNL mass.

You will take a sample of such decays and push it through the whole chain in figure 8.1: transport the two daughters through the detector, look at the result in the event display, turn energy deposits into detector hits, and finally form an illustrative weighted acceptance estimate.

8.2. The packages

Package	Repository	Role
Geometry	`ShipSoft/Geometry`	The SHiP detector described in GeoModel; writes `ship_geometry.db`.
geometry service	`ShipSoft/geometry_service`	Framework-independent access to that geometry, as GeoModel or as Geant4 volumes.
field service	`ShipSoft/field_service`	Field maps in a versioned file format, evaluated for Geant4 and ACTS.
data model	`ShipSoft/data-model`	The event classes (`MCParticle`, `SimHit`, ...) and their ROOT I/O.
ship-random	`ShipSoft/ship-random`	The counter-based random number generator shared by all packages.
aegir	`ShipSoft/aegir`	Simulation: event sources plus Geant4, as Phlex plugins.
sea-cucumber	`ShipSoft/sea-cucumber`	Event display, in ROOT REve or in the browser.
Shannon	`ShipSoft/Shannon`	Digitisation: simulated hits into detector hits.
Trout	`ShipSoft/Trout`	Reconstruction (not covered hands-on, see chapter 17).

All of these are new code. **FairShip**, the framework used for SHiP so far, is not needed for anything in this booklet.

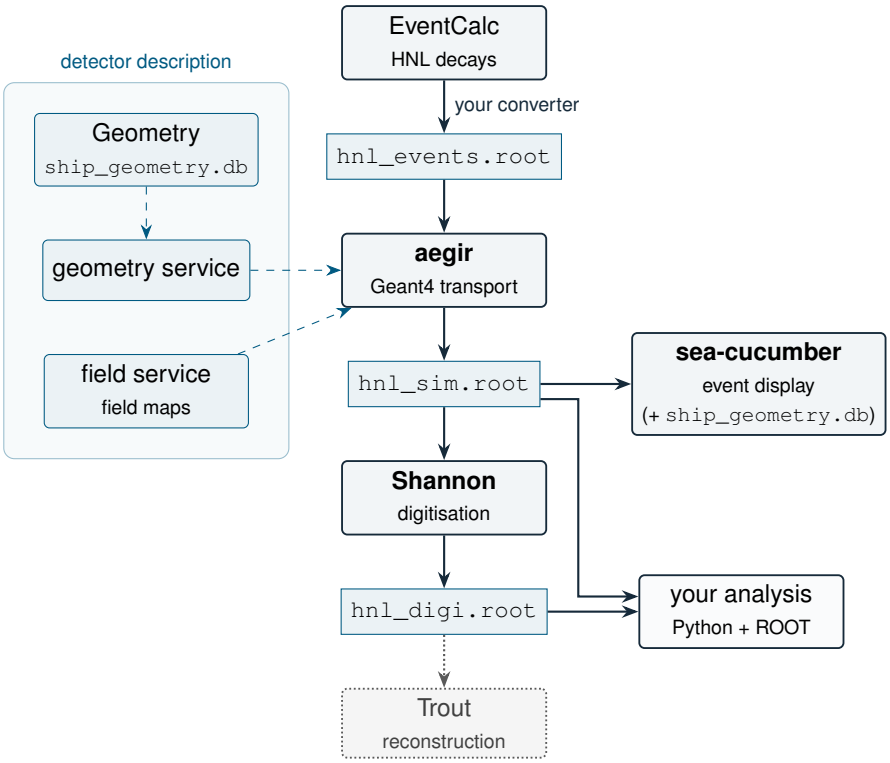


Figure 8.1.: The chain followed in this booklet. Solid arrows carry event data between stages; dashed arrows supply detector information. The dotted Trout branch is an outlook. Event files are ROOT RNTuples; the geometry is a SQLite database and field maps are ROOT RNTuples in their own schema.

8.3. Phlex in one page

aegir, **Shannon** and **Trout** are not programs in their own right: they are collections of plugins for Phlex, a data-flow framework developed by the Framework R&D project. You never write a `main()`. Instead you write a *workflow* file that names the plugins to load and configures each one, and the `phlex` executable builds a graph from it and runs it, in parallel where the data allow.

Plugins. Each plugin is a shared library found on `PHLEX_PLUGIN_PATH`, referred to in a workflow by its `cpp` name. There are four kinds you will meet:

driver says which data cells exist: for example “a job with 100 events” (`generate_layers`).

source / provider puts products into a cell: generated or read particles per event, the geometry and field once per job.

transform turns products into new products, such as particles into hits (Geant4) or hits into digitised hits.

observer consumes products without producing any: output writers and histogram fillers.

Layers. Cells are organised in layers: a `job` contains `events` in **aegir**, and `spills` in **Shannon**. A product lives in one layer; the geometry is a job-level product shared by every event.

Stages. Every job names its processing stage (`simulation`, `digitisation`), and the products it creates are labelled with it.

Workflows are jsonnet. Jsonnet is JSON with variables, functions and imports. The **aegir** workflows import building blocks from `workflows/lib.libsonnet` and override what they need. Values that change from run to run (number of events, file names) are passed as external variables, so a run is typically:

```
phlex -c <(jsonnet --ext-str events=100 --ext-str infile=hnl_events.root \
                   my_workflow.jsonnet) -j 4
```

`jsonnet` expands the workflow to plain JSON, the shell hands it to `phlex` as a file, and `-j 4` lets Phlex use four threads.

Note

You can always look at what `phlex` actually receives by running the `jsonnet` part on its own. This is the first thing to do when a workflow does not behave as expected.

8.4. Reproducibility in one paragraph

The simulation and digitisation plugins use **ship-random**'s `PhiloxRng`, a counter-based generator: a fresh generator is seeded per event from *(seed, stream key, event number)*, with no state shared between events. The same seed therefore gives the same events whatever the number of threads and in whatever order events finish. The aegir generator and transport plugins choose and log a random seed when none is supplied; Shannon defaults to seed zero. This booklet sets explicit seeds. Reproducibility also requires fixed inputs, software and configuration. Shannon seeds by input entry/spill index, so reordering a simulation file changes which random stream a physical event receives.

Pitfall

Output writers fill RNTuples from several threads at once, so entries appear in the order events *finish*, not in event-number order. In **aegir** output every entry carries its `event_number`; never assume that entry i is event i.

Chapter 9.

The detector description

Where you are

Step: 3 of 11 in Part II: the first package of the chain.

You need: chapter 8.

You will be able to: build `ship_geometry.db`, find any volume and its position, and name the volumes that will record hits.

9.1. GeoModel in a nutshell

The SHiP detector is described with GeoModel, the geometry toolkit developed for ATLAS. A GeoModel description is ordinary C++ that builds a tree of volumes:

- a *shape* (`GeoBox`, `GeoTubs`, `GeoTrap`, ...) gives the extent;
- a *logical volume* (`GeoLogVol`) adds a name and a material;
- a *physical volume* (`GeoPhysVol`, or `GeoFullPhysVol` when its global position must be retrievable) places a logical volume, with a transform, inside its mother.

The tree is built once and written to a single SQLite file, `ship_geometry.db`. Every downstream package (simulation, event display, reconstruction) reads that file, so they all see exactly the same detector.

In `ShipSoft/Geometry` each subsystem is a factory class in `subsystems/<Name>/` whose `build()` returns the subsystem's top volume; `SHiPGeometryBuilder` creates the world (the cavern) and places each subsystem at its z position. Materials are defined once, in `SHiPMaterials`.

9.2. Coordinates, units and names

Origin	front face of the target (the first tungsten disk)
z	along the beam, positive downstream
y	vertical, pointing up
x	horizontal, completing a right-handed system
Units	millimetres and radians (GeoModel's native units)
Beam axis	1.7 m above the cavern floor

Volume names follow a path convention: `/SHiP/<subsystem>/<part>`, for example `/SHiP/trackers/straw_gas` or `/SHiP/timing_detector/bar`. Downstream code selects volumes by matching on these names, so they matter: you will use them in chapter 13 to tell Geant4 which volumes are sensitive.

9.3. What is implemented

Subsystem	Status	Content
Cavern	complete	world volume with the rock cavities subtracted
Target	complete	33 tungsten disks in a grooved steel core, jacket and endcap
Muon shield	approximate	6 stations; box approximations of the real shapes
Neutrino detector	approximate	veto, Si/W target, HCAL with individual fibres
Magnet	approximate	iron yoke, box-shaped coils
Decay volume	implemented	frustum: SBT steel structure, liquid-scintillator (LAB) cells, helium
Upstream tagger	simulation-ready	4.4×6.4 m plane of 47 000 polystyrene tiles
Trackers	complete	4 stations $\times$ 4 stereo views, 9600 straws
Timing detector	complete	330 scintillator bars
Calorimeter	simulation-ready	ECAL and HCAL sampling layers

The status column is taken from the `ShipSoft/Geometry` README, where each subsystem links to its own README with the volume tree and materials. The nominal envelope of every subsystem (where it starts and ends in z, how wide it is) is tabulated in `subsystem_envelopes.csv` in the same repository.

9.4. Building the geometry

Exercise 9.1: Build `ship_geometry.db`

In your `Geometry` checkout:

```
pixi run geometry
ls -lh build/ship_geometry.db
```

The `geometry` task compiles the package and runs `build/apps/build_geometry build/ship_geometry.db`. Copy the result to your working directory, where all later chapters expect it:

```
cp build/ship_geometry.db "$SHIP_TUTORIAL/work/"
```

Note

The `shipgeometry` conda package installs `build_geometry` too, so in the **aegir** environment `pixi run build_geometry` writes the same file into the current directory without compiling anything. Building from the `ShipSoft/Geometry` checkout is what you do once you change the description.

Exercise 9.2: Validate it

```
pixi run ./build/apps/validate_geometry build/ship_geometry.db
```

The validator checks the database structure and prints the volumes, the materials and the positions it finds. Skim the material list: which material fills the decay volume, and which fills the straws?

9.5. Looking at the geometry

`gmex`, the GeoModel explorer, opens a `.db` file directly:

```
pixi run gmex build/ship_geometry.db
```

In the volume tree on the left, expand `/SHiP`, tick the subsystems you want to see and use the mouse to rotate and zoom.

Exercise 9.3: Find the decay volume

Open the geometry in `gmex`. Show only the decay volume, the trackers and the calorimeter. Roughly where in z does the decay volume start and end? Then check your estimate against `subsystem_envelopes.csv`.

The event display can print the same information as text, which is quicker for scripting:

```
cd "$SHIP_TUTORIAL/sea-cucumber"
pixi run sc --geometry ../work/ship_geometry.db --inspect 4
pixi run sc --geometry ../work/ship_geometry.db --inspect 6 --inspect-match "
    ↪ *trackers*"
```

The number after `--inspect` is the depth of the tree to walk; `--inspect-match` restricts the listing to names matching a pattern.

Exercise 9.4: The sensitive volumes

In its default scoring mode, Geant4 records positive-energy-deposit steps only in volumes declared sensitive, and in chapter 13 you will declare them by name. Using `--inspect`, `gmex`, or the subsystem READMEs, find a name fragment that selects the active material of each of these five detectors. Decide whether a single substring can select it without also selecting passive material:

(a) the upstream background tagger (UBT) tiles;

(b) the surround background tagger (SBT) liquid-scintillator cells;

(c) the gas of the straw tubes;

(d) the calorimeter's scintillator layers;

(e) the timing-detector bars.

Chapter 10.

The geometry and field services

Where you are

Step: 4 of 11 in Part II.

You need: chapter 9 (`ship_geometry.db`); the field maps from chapter 7.

You will be able to: explain how other packages reach the geometry and the magnetic field, and inspect, evaluate and plot a field map.

The geometry file and the field maps are just data. Two small libraries turn them into something a simulation or reconstruction can use, without tying either to a particular framework: neither library includes a Phlex header, so these libraries can serve simulation, reconstruction and standalone programs. The current **sea-cucumber** implementation opens GeoModelIO directly; its optional geometry-service binding is still a stub.

10.1. The geometry service

`ShipSoft/geometry_service` wraps the GeoModel description behind one abstract interface, modelled on DD4hep's `IGeoSvc` (LHCb, Key4hep) and ALICE O^2's `GeometryManager`:

```
namespace ship {
class IGeometryService {
 public:
  virtual ~IGeometryService() = default;
  virtual const GeoVPhysVol* geoModelWorld() const = 0;           // GeoModel
    ↪ tree
  virtual G4LogicalVolume* geant4WorldLogical() = 0;              // Geant4
    ↪ world
  virtual G4LogicalVolume* getLogicalVolume(const std::string& name) const =
    ↪ 0;
};
}
```

`SHiPGeometryService` implements it, with two ways to obtain the geometry:

```
#include <GeometryService/SHiPGeometryService.h>

// from a file written by build_geometry
auto svc = ship::SHiPGeometryService::fromFile("ship_geometry.db");
// or straight from the C++ description, without a file
auto svc2 = ship::SHiPGeometryService::fromSource();

const GeoVPhysVol* world = svc->geoModelWorld();
```

Two design decisions are worth knowing because they explain error messages you may meet:

The GeoModel tree is built eagerly.
Both factory functions construct the full tree immediately, so a broken geometry fails at start-up, not in the middle of a run.

The Geant4 conversion is lazy.
`geant4WorldLogical()` converts GeoModel to Geant4 volumes on its first call only (guarded by `std::call_once`), because the conversion needs a `G4RunManager` on the calling thread. Call it from `G4VUserDetectorConstruction::Construct()`. `getLogicalVolume()` only works after that.

The library also provides `G4RayScanner`, which walks a straight line through the Geant4 geometry and returns the materials crossed with their lengths (useful for material-budget studies), and `GeometryThread`, which funnels all Geant4 geometry creation through one thread, as Geant4's multi-threading model requires.

Exercise 10.1: How aegir finds the geometry

Open `aegir/src/geometry_geomodel_provider.cpp`. Without running anything, answer:

1. Which two configuration keys does the provider read?
2. The workflow gives `db_file: 'ship_geometry.db'`. Where is the file looked for, and in which order?
3. At which Phlex layer is the geometry published, and why does that make sense?

10.2. The field service

`ShipSoft/field_service` stores magnetic field maps in a versioned file format and evaluates them with covfie (trilinear interpolation on a regular grid). It has four parts, so that each consumer pulls in only what it needs:

Component	Needs	Purpose
core	–	build a field evaluator from an in-memory map
MapIO	ROOT	read and write map files
G4Adapter	Geant4	a `G4MagneticField` for simulation
ActsAdapter	ACTS	an `Acts::MagneticFieldProvider` for reconstruction

10.2.1. The field-map format

A field-map file is a ROOT file holding one RNTuple, `field_maps`, with one entry per map, so a single file can carry several magnets. Each map records its regular grid (first and last node and number of nodes per axis, in mm), the three field components at every node (in tesla), the node ordering, its symmetry, and where the values came from. Reading it needs plain ROOT or uproot, not SHiP libraries. The full specification is `docs/field_map_format.md` in the repository.

Symmetry. A map can be stored for one quadrant only and mirrored across the $x = 0$ or $y = 0$ planes, with a sign (*parity*) per field component and plane. FairShip's quadrant-symmetric muon-shield maps are stored this way.

Frames. A map is stored in the magnet's own frame. Whoever uses it places it in the global frame with a translation. In **aegir**, each magnet is also attached to the logical volumes whose names contain a given pattern; Geant4 evaluates the field only inside those volumes, so field-free drift regions cost nothing:

```
field: lib.field_map([
  { name: 'MuonShield',   volume_pattern: '/SHiP/muon_shield',
    file: 'muon_shield.root',         map: 'muon_shield',
    translation: [0, 0, 2040] },       // shield entrance [mm]
  { name: 'Spectrometer', volume_pattern: '/SHiP/magnet',
    file: 'spectrometer_dipole.root', map: 'spectrometer_dipole',
    translation: [0, 0, 89570] },      // magnet centre [mm]
]),
```

10.2.2. The command-line tools

`fieldmap_dump f.root`	print the metadata of every map in the file
`fieldmap_dump f.root map`	read `x y z` lines (mm) from standard input, print `x y z Bx By Bz` (mm, T)
`plot_field_map f.root map out.root`	plot B_y along the beam axis and in the xz, yz and central xy planes, as ROOT histograms and one PDF per plot
`generate_constant_fieldmap`	write a uniform field over a box, for tests
`fairship_to_fieldmap`	convert a FairShip map; `--symmetry quadrant_dipole` for the quadrant-symmetric ones

The tools are in the `field-service-tools` conda package; from a `ShipSoft/field_service` checkout, `pixi run install` puts them on the environment's `PATH`. **[TBD: confirm the tutorial environment includes them]**

Exercise 10.2: Inspect the field maps

In `work/`, where the provided maps are:

```
fieldmap_dump spectrometer_dipole.root
fieldmap_dump muon_shield.root
```

For each map, note the grid extent and spacing, and whether it is mirrored.

Exercise 10.3: Evaluate and plot the spectrometer field

The spectrometer map is stored in the magnet's own frame, so its centre is at $(0, 0, 0)$. Evaluate the field there and one metre either side along the beam:

```
printf '0 0 0\n0 0 -1000\n0 0 1000\n' | \
  fieldmap_dump spectrometer_dipole.root spectrometer_dipole
plot_field_map spectrometer_dipole.root spectrometer_dipole spectro_plots.
   ↪ root
```

Which component dominates? Open the B_y-versus-z PDF and estimate the field integral $\int B_y \,\mathrm{d}z$. The transverse momentum kick is $\Delta p_T\,[\mathrm{GeV}] \approx 0.3 \int B \,\mathrm{d}z\,[\mathrm{T\,m}]$: what does that mean for the bending of a 5 GeV muon from an HNL decay?

Pitfall

Outside its grid the evaluator clamps to the boundary value. A flat plateau beyond the edges of a plot is an artefact of the plotting, not a physical fringe field. The same clamping applies in the simulation, which evaluates the field only inside the volumes matching `volume_pattern`: check that the map's grid covers those volumes, or Geant4 sees the boundary value continued up to the volume edge.

Note

For a demonstration, a uniform spectrometer field can replace the real map. For example, 1 T vertical over the magnet volume:

```
generate_constant_fieldmap spectrometer_dipole.root spectrometer_dipole \
  0 1 0  -2500 2500  -3500 3500  -2500 2500
```

(arguments: file, map name, $B_x\, B_y\, B_z$ in T, then $x_{\min}\, x_{\max}\, y_{\min}\, y_{\max}\, z_{\min}\, z_{\max}$ in mm). The workflow still references `muon_shield.root`: either supply that map too, or remove its magnet entry for this spectrometer-only demonstration. The uniform-field sample is not suitable for an absolute performance prediction.

Chapter 11.

The event data model

Where you are

Step: 5 of 11 in Part II.

You need: chapter 8; the HNL input file from chapter 7.

You will be able to: name the event classes and their units, describe the layout of each kind of file, and open one from Python.

11.1. Plain structs, written as RNTuples

`ShipSoft/data-model` defines the classes every package exchanges. They are deliberately simple: plain C++ structs with default values, no methods to speak of, no pointers between objects, and ROOT dictionaries so that they can be written to and read from ROOT files. Relations are expressed by *indices* (a mother index into the same particle vector, a Geant4 track ID), never by pointers, which keeps the files readable from any language.

The files are written as RNTuples, ROOT's newer columnar I/O format, designed for efficient reading and writing, with with a stable on-disk format since ROOT 6.34. An RNTuple has named *fields*; a field can be a number, a struct or a `std::vector` of structs, and every member of a struct is itself a (sub)field.

Dimensional floating-point quantities use `double`; IDs and statuses are integers. The storage units are: **mm, GeV, ns**. The headers also provide typed views (`SHiP/QuantityView.hpp`) that attach physical units at compile time; **aegir** uses them internally, but you do not need them to read a file.

11.2. The classes in this tutorial

Class	Produced by	Content
`EventHeader`	generator / converter	event `weight`, `original_event_id`
`MCParticle`	generator / converter	generator record: `pdgCode`, `vertex`, `momentum`, `energy` (total), `time`, `motherId`, `mothers`, `status`
`SimParticle`	**aegir** (Geant4)	every tracked particle: `trackId`, `parentId`, `pdgCode`, `vertex`, `endpoint`, `momentum`, `energy` (kinetic), `time`, `creatorProcess`
`SimHit`	**aegir** (Geant4)	one positive-energy-deposit step in a sensitive volume (default scoring mode): `detectorId`, `geometryNodeId`, `trackId`, `pdgCode`, `position`, `momentum`, `energyDeposit`, `time`, `pathLength`
`RecHit`	**Shannon**	a digitised hit; same members as `SimHit` minus `geometryNodeId`
detector hits	**Shannon**	`UBTHit`, `SBTHit`, `StrawTubesHit`, `CaloHit`, `TimeDetHit`: one per detector, each wrapping a `RecHit` as `recHit`

Appendix B lists every member with its unit, and the `RecParticle` and `TrackFitResult` classes used by reconstruction.

11.2.1. Generator particles

`MCParticle::status` follows the HepMC convention. Generator adapters must map their native status codes to it; Pythia 8 internal status codes should not be copied unchanged. What matters here:

`status`	meaning
1	final state: not decayed by the generator. **Only these are given to Geant4.**
2	decayed by the generator
4	incoming beam particle

Mother links point into the same vector: `motherId` is the index of the first mother, or -1; `mothers` lists all of them. For our HNL events the vector is short, as chapter 12 shows: the HNL with status 2 and its two daughters with status 1, pointing back to it.

Pitfall

`MCParticle::energy` is the *total* energy; `SimParticle::energy` is the *kinetic* energy. Compute invariant masses from `MCParticle`s, or recompute E from $|\vec{p}|$ and the mass.

11.2.2. Simulated particles and hits

Every particle Geant4 tracks becomes a `SimParticle`. Its `trackId` is the Geant4 track ID, unique *within one event*; the particles that came from the generator have `parentId` $= 0$, and every secondary points to the track that created it. `creatorProcess` is the Geant4 process sub-type that created it.

In the default scoring mode, a Geant4 step inside a sensitive volume becomes a `SimHit` only if its deposited energy is positive. Crossing mode records volume entrances and has a different meaning. Its `trackId` says which particle made it, and its `detectorId` says which detector it is in. The detector numbering is fixed by the data model:

`detectorId`	detector
0	upstream background tagger (UBT)
1	surround background tagger (SBT)
2	straw tubes
3	calorimeter
4	timing detector

Pitfall

aegir assigns `detectorId` as the *position* of the matching name in its `sensitive_volumes` list, while **Shannon** interprets it with the table above. An ID outside 0–4 raises "No digitiser registered for detector ID"; a wrong ordering within 0–4 silently calls the wrong digitiser. The list in the simulation workflow must therefore have exactly five entries, in this order (section 13.3).

11.3. What the files look like

Three kinds of file appear in this tutorial. All of them hold their data in RNTuples inside an ordinary ROOT file.

Generator input (`hnl_events.root`, from the converter). One RNTuple, `events`, one entry per event:

field	type
`mc_particles`	`std::vector<SHiP::MCParticle>`
`event_header`	`SHiP::EventHeader`

Simulation output (`hnl_sim.root`, from **aegir**). One RNTuple, `events`, one entry per event:

field	type
`event_number`	`std::uint64_t`
`event_header`	`SHiP::EventHeader`
`mc_particles`	`std::vector<SHiP::MCParticle>`
`sim_hits`	`std::vector<SHiP::SimHit>`
`sim_particles`	`std::vector<SHiP::SimParticle>`

Digitisation output (`hnl_digi.root`, from **Shannon**). Seven RNTuples, one per collection, each with a single field `hit` and *one entry per object* (a hit, or a particle for `sim_particles`): `ubt_hits`, `sbt_hits`, `straw_tubes_hits`, `calorimeter_hits`, `timing_detector_hits`, and copies of the input as `sim_hits` and `sim_particles`. Chapter 15 discusses what this layout means for an analysis.

Exercise 11.1: Look inside the HNL input file

In `work/`, start Python in the **aegir** environment and open the file you were given:

```
import ROOT
reader = ROOT.RNTupleReader.Open("events", "hnl_events.root")
reader.PrintInfo()      # fields, types, number of entries, compression
reader.Show(0)          # the first entry, as JSON
```

How many events does the file hold? How many particles are in the first event, what are their PDG codes and statuses, and which ones will Geant4 track?

Note

ROOT reads these files through the data-model dictionaries, which the pixi environment makes available automatically; that is what lets you write `p.pdgCode` on a particle in Python. Outside the environment, uproot can read the same members column by column without any SHiP library.

11.4. Keeping old files readable

Backward readability is a project goal checked with frozen reference files and a committed schema snapshot, but it has known exceptions. In 0.6, reads of pre-0.5 fixtures are marked as expected failures for a ROOT issue. The detector-wrapper rename `recHit` to `rec_hit` has no working conversion rule: reading an old wrapper intentionally raises a fatal error. A successful read of a simulation fixture therefore does not demonstrate that old Shannon output is readable. Keep the complete tutorial chain on 0.5 until an explicit migration and readback test succeeds. See `data-model/tests/data/README.md` at the revision in appendix D.

Chapter 12.

The HNL sample

Where you are

Step: 6 of 11 in Part II.

You need: chapter 11.

You will be able to: read EventCalc's conventions (weights, frames, units), and check an input sample before simulating it.

12.1. EventCalc

`ShipSoft/EventCalc` generates decays of long-lived particles inside the SHiP decay volume and computes how many such decays to expect. It does not simulate the detector: it produces truth-level momenta and decay positions, which is exactly what a detector simulation needs as input. For every mass m and proper decay length $c\tau$ it

1. interpolates tabulated production probabilities and angle–energy distributions of the long-lived particle;
2. samples momenta pointing towards the decay volume;
3. samples decay positions from the exponential decay law along the volume, and applies its transverse shape;
4. generates the decay in the rest frame from the matrix element (hadronising with Pythia 8 or exHad where needed) and boosts it to the laboratory;
5. combines production, geometric acceptance, decay probability and branching ratio into the expected number of decays.

For the HNL the input is the mixing pattern $(\xi_e, \xi_\mu, \xi_\tau)$ with $\xi_e + \xi_\mu + \xi_\tau = 1$ and $U_\alpha^2 = U^2 \xi_\alpha$. The tutorial sample uses pure muon mixing and the two charge-conjugate channels, `Pimu` ($\pi^+\mu^-$) and `Pimubar` ($\pi^-\mu^+$). A launch card for such a sample looks like this:

code/eventcalc/hnl_pimu.json

```
{
  "model": "HNL",
  "mixing_pattern": [0.0, 1.0, 0.0],
  "masses": [1.0],
  "c_taus": [100.0],
  "decay_channels": ["Pimu", "Pimubar"],
  "events": 10000,
  "seed": 20261015,
  "plots": false,
  "export_events": true
}
```

```
python3 simulate.py --card "$CODE/eventcalc/hnl_pimu.json"
```

Note

The card requests 10 000 events (mass 1 GeV, $c\tau = 100$ m); this does not establish the provenance of the distributed ROOT file. Record the actual sample settings: **[TBD: mass, $c\tau$, number of events, EventCalc version]**. You do not need to run EventCalc during the tutorial; the card above is for reference.

12.1.1. What comes out

EventCalc writes a text file per mass–lifetime point. Its header line gives the numbers needed for a yield: the total number of HNLs produced, the polar and azimuthal acceptance, the mean decay probability, the visible branching ratio and their product, the expected number of decays N_{exp}. Each event row then holds the HNL four-momentum, mass, decay probability P_{decay} and decay position, followed by the daughters' four-momenta, masses and PDG codes. Two conventions matter:

- **Every event has a weight**, $w = P_{\text{decay}}$, the probability that this HNL decays inside the volume. Distributions must be filled with it, and the expected number of decays satisfying a selection is

$$N_{\text{sel}} = N_{\text{exp}} \frac{\sum_{i \in \text{sel}} w_i}{\sum_i w_i}.$$

- **Positions are in metres, measured from the centre of the target**, while the SHiP geometry uses millimetres from its front face (section 9.2).

12.2. From EventCalc to the data model

aegir reads generator events from a ROOT file in the SHiP data model (section 11.3). The converter used for the tutorial sample **[TBD: name, location, author]** produces, for each EventCalc event, one entry of the `events` RNTuple with:

- `mc_particles`: the HNL with status 2, followed by its daughters with status 1 and `motherId` $= 0$. Daughters carry the decay position as their `vertex`, in mm in the geometry's frame, and their total energy in GeV;
- `event_header`: `weight` $= P_{\text{decay}}$ and `original_event_id` set to the original generator index. Keep `mothers` consistent with `motherId`: daughters have `mothers` $= \{0\}$, and the motherless HNL has an empty list.

Exercise 12.1: Check the input before simulating it

A wrong frame or a missing weight is much cheaper to find now than after the simulation. Run the checker in `work/`:

```
python $CODE/analysis/check_input.py hnl_events.root
```

1. Which particles will Geant4 track, and do all of them have a mother?
2. Is there an `event_header`? What is the range of the weights?
3. What fraction of decay vertices lies outside the nominal longitudinal range? What would a fraction close to 100% tell you?

Checkpoint

Only pion–muon pairs $(211, 13)$ or $(-211, -13)$ are tracked, none without a mother; the header is present and the weights lie between 0 and 1; the checker reports how many vertices pass $32.92 < z < 83.32$ m. This is a longitudinal check, not full containment.

The script is short; read it. It shows the basic pattern for reading these files from Python, which you will use again in chapter 16:

code/analysis/check_input.py

```
    for i in range(n_events):
        vertex_z = None
        event = particles(i)
        for p in event:
            statuses[p.status] += 1
            if p.status != 1:
                continue
            tracked_pdg[p.pdgCode] += 1
            mid = int(p.motherId)
            if mid < 0:
                orphans += 1
            elif mid >= len(event):
                invalid_mothers += 1
            else:
                vertex_z = float(p.vertex[2])
            mothers = list(p.mothers)
            if (mid < 0 and mothers) or (mid >= 0 and (not mothers or mothers
↪ [0] != mid)):
                inconsistent_links += 1
        if vertex_z is not None:
            decay_z.append(vertex_z)
        w = float(headers(i).weight) if headers else 1.0
        if not math.isfinite(w) or not 0 <= w <= 1:
            raise ValueError(f"Entry {i}: expected P_decay in [0, 1], got {w}
↪ ")
        weights.append(w)
```

Chapter 13.

Simulation with aegir

Where you are

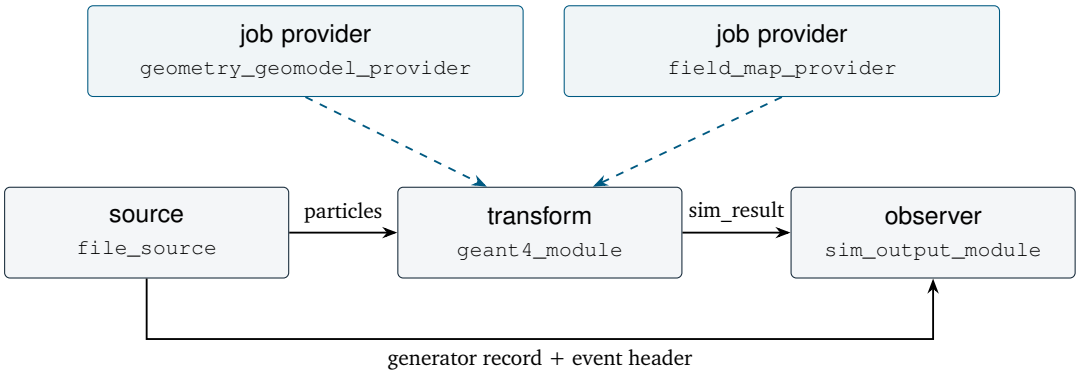
Step: 7 of 11 in Part II: the first simulation.

You need: chapters 9, 10 and 12: the geometry, the field maps and a checked HNL sample.

You will be able to: write and run a simulation workflow, choose the sensitive volumes, read the validation histograms, and reproduce a run exactly.

13.1. What aegir is

`ShipSoft/aegir` is the SHiP simulation: a set of Phlex plugins that generate or read particles, transport them through the detector with Geant4, and write the result. A simulation job is a graph of four kinds of node:



Sources work per event; the geometry and the field are providers at the job level, built once and shared by all events. The plugins available are:

Plugin	Kind	Purpose
`particle_gun_source`	source	single particles in a cone
`pythia8_source`	source	Pythia 8 fixed-target collisions
`fixed_target_source`	source	Pythia 8 on tungsten, with the interaction point sampled in the target
`genie_reader_source`	source	pre-generated GENIE neutrino interactions
`file_source`	source	**events read from a data-model file (our case)**
`geometry_builtin_provider`	provider	a test geometry (tungsten block and silicon planes)
`geometry_gdml_provider`	provider	geometry from a GDML file
`geometry_geomodel_provider`	provider	**the SHiP geometry via the geometry service**
`field_null_provider`	provider	no field
`field_map_provider`	provider	**field maps via the field service**
`geant4_module`	transform	Geant4 transport, multi-threaded
`sim_output_module`	observer	RNTuple output and validation histograms

13.2. The HNL workflow

The workflow for this tutorial is `code/workflows/hnl_sim.jsonnet`. The workflows shipped with **aegir** (in its `workflows/` directory) build jobs from the blocks in `lib.libsonnet`; this one spells everything out so that you can read the whole job in one place:

code/workflows/hnl_sim.jsonnet

```
local stage = 'simulation';

function(seed=20261015) {
  stage: stage,

  // One job containing `events` events. Must not exceed the number of
   ↪ entries
  // in the input file.
  driver: {
    cpp: 'generate_layers',
    layers: { event: { total: std.parseInt(std.extVar('events')) } },
  },

  sources: {
    // The HNL events: one entry of the input RNTuple per event. The tracked
    // particles are the status-1 entries of `mc_particles`; the event weight
    // comes from the `event_header` field.
    input: {
      cpp: 'file_source',
      stage: stage,  // sources need their own copy of the stage name
      input_file: std.extVar('infile'),
      ntuple: 'events',
      product: 'mc_particles',
      skip: 0,  // first entry to read
    },

    // The SHiP geometry, read through the geometry service. The position of
   ↪ a
    // name fragment in `sensitive_volumes` becomes SimHit::detectorId, and
    // Shannon reads that ID with the data model's detector_id enum, so this
    // list must have exactly these five entries in exactly this order. A
    // logical volume is sensitive if its name *contains* the fragment.
    geometry: {
      cpp: 'geometry_geomodel_provider',
      db_file: 'ship_geometry.db',
      sensitive_volumes: [
        '/SHiP/upstream_tagger/',          // 0 UBT (tiles, plus their air
   ↪ envelopes)
        '/SHiP/decay_volume/sbt/sensors',  // 1 SBT (LAB cells, plus their
   ↪ walls)
        '/SHiP/trackers/straw_gas',        // 2 straw tubes
        '/SHiP/calorimeter/',              // 3 calorimeter (scintillator and
   ↪  absorber)
        '/SHiP/timing_detector/bar',       // 4 timing detector
      ],
    },

    // Field maps in the SHiP field-map format. Each map is stored in its own
    // frame; `translation` (mm) places its origin in the geometry, and the
    // field is attached to the logical volumes whose names contain
```

```
    // `volume_pattern`. Bare file names are looked up in the current
    // directory, then in $SHIPFIELD_ROOT/share/field/.
    field: {
      cpp: 'field_map_provider',
      magnets: [
        {
          name: 'MuonShield',
          volume_pattern: '/SHiP/muon_shield',
          file: 'muon_shield.root',
          map: 'muon_shield',
          translation: [0, 0, 2040],  // shield entrance
        },
        {
          name: 'Spectrometer',
          volume_pattern: '/SHiP/magnet',
          file: 'spectrometer_dipole.root',
          map: 'spectrometer_dipole',
          translation: [0, 0, 89570],  // magnet centre
        },
      ],
    },
  },

  modules: {
    // Geant4 transport of the status-1 particles through geometry and field.
    geant4: {
      cpp: 'geant4_module',
      physics_list: 'FTFP_BERT',
      verbosity: 0,
      seed: seed,
      progress_interval: 100,  // log every N events
    },

    // 'full' writes event_number, event_header, mc_particles, sim_hits and
    // sim_particles per event, plus validation histograms to `histo_file`.
    output: {
      cpp: 'sim_output_module',
      mode: 'full',
      rntuple_file: std.extVar('simout'),
      histo_file: std.extVar('histo'),
    },
  },
}
```

13.3. Sensitive volumes

In default scoring mode, Geant4 creates a `SimHit` for every positive-energy-deposit step a particle takes inside a *sensitive* volume. **aegir** makes a logical volume sensitive when its name *contains* one of the fragments in `sensitive_volumes`, and records the *index* of that fragment as `detectorId`. Three consequences:

1. The order of the list is the detector numbering. **Shannon** reads `detectorId` with the data model's `detector_id` enum (chapter 11), so the list must be UBT, SBT, straws, calorimeter, timing detector, and nothing else.

2. A fragment selects everything whose name contains it, including passive volumes with similar names. In this workflow the UBT entry also catches the air envelopes around the tiles, the SBT entry the aluminium walls of the cells, and the calorimeter entry the lead and iron absorbers (section 9.5).

3. Volumes that match no fragment produce no hits at all. If a detector is missing from the list, it is silently invisible.

Note

The **aegir** workflows that use the SHiP geometry ship with `sensitive_volumes: ['ScoringPlane']`, which matches nothing in the real detector: they are set up for muon-flux studies at scoring planes, not for detector hits. The demonstration list in `hnl_sim.jsonnet` preserves Shannon's numbering, but does not select active material exclusively.

Pitfall

The broad UBT, SBT and calorimeter prefixes also record passive-material energy deposits. Shannon smears them as if they were detector hits; the example analysis cannot distinguish them using detector ID alone. The veto and calorimeter response are therefore teaching proxies, not physical predictions. A production workflow needs active-volume selection with an explicit detector-ID mapping, or a geometry-aware hit filter.

13.4. Running it

Exercise 13.1: Simulate 100 HNL decays

Open a shell in the **aegir** environment and go to your working directory, which must contain `ship_geometry.db`, the two field maps and `hnl_events.root`:

```
cd "$SHIP_TUTORIAL/aegir"
pixi shell
cd "$SHIP_TUTORIAL/work"
time phlex -j 4 -c <(jsonnet \
    --ext-str events=100 \
    --ext-str infile=hnl_events.root \
    --ext-str simout=hnl_sim.root \
    --ext-str histo=hnl_sim_validation.root \
    $CODE/workflows/hnl_sim.jsonnet)
```

Watch the log: the geometry is read and converted, the field maps are loaded and attached to their volumes, the Geant4 seed is printed, and then the event loop reports progress every 100 events.

Checkpoint

Two new files, `hnl_sim.root` and `hnl_sim_validation.root`, and no error. With the Python lines from section 11.3, `hnl_sim.root` shows 100 entries in the `events` RNTuple with the five fields of section 11.3.

Pitfall

The driver creates exactly `events` events whether or not the input has that many entries. Past the end of the file, `file_source` publishes *empty* events with weight 1. Always ask for at most as many events as the input holds (section 12.2 printed the number).

Exercise 13.2: Read the validation histograms

aegir fills a few histograms as it runs. Open them with a ROOT browser:

```
root -l hnl_sim_validation.root
root [1] new TBrowser
```

1. `h_hit_detector`: which detector IDs occur, and roughly in which proportions? Is any of the five missing?
2. `h_hit_z`: identify the straw stations, the timing detector and the calorimeter. Are there hits upstream of the trackers, and from which detector?
3. `h_mc_multiplicity`: every input event holds three generator particles. Why does this histogram say two?

13.5. Reproducibility and threads

Exercise 13.3: Same seed, same events

Rename your output, run the same command again, and compare selected per-event summaries with the summary script from chapter 16, or simply compare the validation histograms. Then run once more with `-j 1` and once with `--tla-code seed=1` added to the `jsonnet` command.

```
python $CODE/analysis/sim_summary.py hnl_sim.root hnl_sim_run2.root
```

Which summaries agree exactly, and which differ? Equal summaries are a smoke check, not proof that every field in the two files is identical.

13.6. Going further

Exercise 13.4: Optional: switch the field off

Make a copy of the workflow that uses `field_null_provider` (no configuration keys) instead of `field_map_provider`, simulate the same events into `hnl_sim_nofield.root`, and keep the file: you will look at the difference in the event display (chapter 14) and in the analysis (chapter 16).

Exercise 13.5: Optional: full statistics

Simulate the whole input file in the background while you continue with the next chapters. With `-j` set to the number of cores of your machine this takes **[TBD: time for the provided sample]**.

Chapter 14.

Looking at events with sea-cucumber

Where you are

Step: 8 of 11 in Part II.

You need: chapter 13: `hnl_sim.root`; `ship_geometry.db`.

You will be able to: look at simulated events in the REve and web displays, and write a view configuration of your own.

14.1. Three inputs and nothing else

`ShipSoft/sea-cucumber` takes a GeoModel `.db`, a data file in the SHiP data model, and optionally a view configuration in TOML. It links no simulation or analysis code, so it can inspect supported event-oriented files regardless of their producer; it does not support every on-disk layout. From the `events` RNTuple it reads the fields `mc_particles`, `sim_hits`, `sim_particles` and `rec_particles`, and tolerates any of them being absent.

It has two front ends sharing one core:

the REve display
: ROOT's web-based event display, one event at a time, started with `pixi run sc`;

the web display
: a three.js page in your browser: the C++ side first writes the geometry and events to JSON files, which the page then renders. You can arrange any number of zoomed views and save the layout.

Following the ALICE O^2 event-display design, the C++ side does all the interpretation (reading the geometry, choosing colours, tessellating shapes), and the viewers only draw.

Note

sea-cucumber reads simulation output. It does not read the per-detector collections written by **Shannon**, so digitised hits cannot be displayed yet.

14.2. The REve display

Exercise 14.1: Your first HNL event

From the `sea-cucumber` directory:

```
pixi run sc --geometry ../work/ship_geometry.db \
            --data ../work/hnl_sim.root \
            --view views/default.toml --event 0
```

REve opens in a browser window. Rotate and zoom the main view. Find the decay vertex (a star marker), the two daughter tracks' hits in the straw stations, and their energy deposits in the calorimeter. Which daughter is the muon?

Checkpoint

A star marker somewhere in the decay volume and two trails of hits downstream of it: four straw stations, the timing detector, and a cluster in the calorimeter.

Pitfall

The geometry has about a million volumes. Reading it takes several seconds at every start, and drawing all of it would make the display unusable, which is why `views/default.toml` excludes the straws, tiles and SBT cells. For repeated use, convert the geometry once into a cache:

```
pixi run geo-cache --geometry ../work/ship_geometry.db --view views/
    ↪ default.toml
pixi run sc --geo-cache geocache --geometry ../work/ship_geometry.db \
            --data ../work/hnl_sim.root --event 0
```

14.3. The web display

Exercise 14.2: All events in the browser

Produce the display files for all events, then serve them:

```
pixi run web-data --geometry ../work/ship_geometry.db \
                  --data ../work/hnl_sim.root \
                  --view views/default.toml --events all
pixi run web
```

Open `http://localhost:8080`. Step through the events from the sidebar. Draw a rectangle on the main view to open a zoomed view of the spectrometer; save the layout.

Note

The web page loads three.js from a public CDN. On a machine without internet access, see the **sea-cucumber** manual for serving it locally. If you work on a remote machine, forward the port: `ssh -L 8080:localhost:8080 <host>`.

14.4. View configuration

A view file has global settings, a `[geometry]` table (which volumes to draw and how), `[hits]` and `[decay]` tables (event styling), and any number of `[[region]]` tables, each a zoomed view with its own window and camera. For example, a side view of the spectrometer:

```
[[region]]
name   = "Spectrometer"
title  = "Spectrometer (side view)"
zmin   = 82000.0          # mm
zmax   = 96000.0
camera = "xz"             # side view, looking along y
max_shapes = 6000
```

Geometry colours are assigned by `[[geometry.style]]` rules, matched as substrings of volume names; the first matching rule wins. The full list of keys is in `docs/MANUAL.md` in the repository.

Exercise 14.3: A view for the HNL

Copy `views/default.toml` to `views/hnl.toml` and edit the copy:

1. The style rules match names like `DecayVolume` or `Tracker`. Compare them with the volume names you found in chapter 9: which rules still apply? Change them to match the `/SHiP/...` names.
2. Add a region showing the decay volume in the horizontal xz projection (viewed along y) (`camera = "xz"`, z from 32 000 to 84 000 mm) with the decay vertex drawn (`draw_decay = true`).

Display a few events with `--view views/hnl.toml`.

Exercise 14.4: Optional: with and without field

If you simulated `hnl_sim_nofield.root` in chapter 13, match the physical event using `event_number`, then display its entry from each file side by side in two browser tabs. Where do the tracks start to differ, and in which direction are they bent? The `--event` option selects an RNTuple entry index, not necessarily the stored `event_number`; multithreaded outputs can have different order.

Chapter 15.

Digitisation with Shannon

Where you are

Step: 9 of 11 in Part II.

You need: chapter 13: `hnl_sim.root`.

You will be able to: digitise simulated hits, explain what the hit-stream output means for an analysis, and test the smearing.

15.1. From energy deposits to detector hits

Geant4 tells us where charged particles deposited energy. A detector reports something else: which channel fired, when, and with what signal, after thresholds, noise, dead time, and the merging of nearby deposits into one channel. *Digitisation* is the step that turns the first into the second, and `ShipSoft/Shannon` is the package that does it for SHiP. Its output provides the hit-stream interface being developed for reconstruction; a realistic electronics response and a real-data adapter are separate tasks.

Shannon is a set of Phlex plugins, like **aegir**:

Plugin	Kind	Purpose
`read_sim_file`	source	reads `sim_hits` and `sim_particles` from **aegir** output
`digitise_hits`	transform	one digitiser per detector, chosen by `detectorId`
`digitised_output_module`	observer	one RNTuple per hit collection, validation histograms

15.2. What the digitisers do today

Every detector has its own digitiser class in `src/detectors/`, so that each detector group can develop its model independently. At the time of writing all five share a placeholder model:

- each `SimHit` becomes exactly one detector hit;
- x and y are smeared with a Gaussian of $\sigma = 0.1\,\text{mm}$ (`gaussian_smearer.hpp`); z, the energy deposit, the momentum and the truth links (`trackId`, `pdgCode`) are copied;
- a time offset is added: **Shannon** places each simulated event at a random time within the fraction of a spill that the simulated protons on target correspond to (the `pot` parameter; a SPS spill is 1.2 s for 4×10^{13} protons).

There are no thresholds, no channel merging, no noise and no inefficiency yet. The structure is the point of this chapter, not the realism.

Note

Random numbers come from **ship-random**: two `PhiloxRng` streams per spill (one for the time offset, one for the smearing), keyed on the spill number. Digitisation is therefore reproducible for a given `seed`, whatever the number of threads for a fixed input entry order. Reordering the simulation entries changes the event-to-random-stream assignment.

15.3. Running it

The workflow is `code/workflows/hnl_digi.jsonnet`:

code/workflows/hnl_digi.jsonnet

```
local stage = 'digitisation';
local layer = 'spill';

{
  stage: stage,

  // Shannon processes "spills". Each spill reads one entry of the simulation
  // file, i.e. here one simulated HNL event.
  driver: {
    cpp: 'generate_layers',
    layers: { [layer]: { parent: 'job', total: std.parseInt(std.extVar('
    ↪ entries')) } },
  },

  sources: {
    // Reads sim_hits and sim_particles from the aegir output. The plugin
    // publishes its products under the creator name `rntuple_source`, which
    // the modules below look for.
    rntuple_source: {
      cpp: 'read_sim_file',
      input_file: std.extVar('infile'),
      ntuple_name: 'events',
      layer: layer,
    },
  },

  modules: {
    // Turns each SimHit into the hit type of its detector, chosen by
    // SimHit::detectorId (0 UBT, 1 SBT, 2 straws, 3 calorimeter, 4 timing).
    digitise_hits: {
      cpp: 'digitise_hits',
      layer: layer,
      seed: 42,
      // Simulated protons on target per spill: sets the time window over
      // which hits are spread (spill length x pot / 4e13). Irrelevant for a
      // signal-only sample; the default is 10000.
      pot: 10000,
    },

    // Writes one RNTuple per hit collection and validation histograms.
    digitised_output: {
      cpp: 'digitised_output_module',
      mode: 'digi',
      rntuple_file: std.extVar('digiout'),
```

```
      histo_file: std.extVar('histo'),
      creator: 'digitise_hits',
      layer: layer,
    },
  },
}
```

Exercise 15.1: Digitise your simulated events

In the **Shannon** environment, from your working directory:

```
cd "$SHIP_TUTORIAL/Shannon"
pixi shell
cd "$SHIP_TUTORIAL/work"
phlex -j 4 -c <(jsonnet \
    --ext-str entries=100 \
    --ext-str infile=hnl_sim.root \
    --ext-str digiout=hnl_digi.root \
    --ext-str histo=hnl_digi_validation.root \
    $CODE/workflows/hnl_digi.jsonnet)
```

Set `entries` to the number of events you simulated.

Checkpoint

`hnl_digi.root` and `hnl_digi_validation.root` exist. In Python, `ROOT.TFile("hnl_digi.root").ls()` lists seven RNTuples, and `h_straw_tubes_multiplicity` in the validation file shows the number of straw hits per event.

Pitfall

"No digitiser registered for detector ID n" means the simulation was run with an out-of-range detector ID. A reordered list using only IDs 0–4 can silently dispatch to the wrong detector model (section 13.3).

15.4. A stream of hits, not a list of events

Shannon writes every collection as its own RNTuple with one entry per hit (section 11.3). This is deliberate. SHiP reads out its detectors continuously, without a trigger: what the experiment records is a stream of time-stamped hits per detector, and grouping hits that belong together into candidate events, by time, is a task of reconstruction. The hit-stream interface is intended for both simulated and real inputs; that design does not make today's placeholder response equivalent to real data.

For an analysis of *simulated* events this has three consequences:

1. **There is no event number.** A hit does not record which simulated event it came from, and the writers fill from several threads, so the order of entries does not follow the events either.
2. **The generator record and the event weight are not copied.** `mc_particles` and `event_header` stay in the simulation file.
3. **Truth links are kept per hit:** `trackId` and `pdgCode` survive in `RecHit`, but track IDs are only unique within one simulated event.

Inclusive distributions over all hits, such as positions, times, energy deposits and resolutions, come from the digitised file. Questions about one simulated event, such as “both daughters crossed four straw stations” or “the SBT stayed silent”, are answered from the simulation file. Chapter 16 is organised accordingly.

Exercise 15.2: Check the smearing

Since **Shannon** copies z, the energy deposit and the truth links, many digitised hits can be paired by a composite key. Repeated keys are ambiguous because there is no event ID; the script excludes them. The script `code/analysis/digi_residuals.py` does this for one collection and histograms the residual $x_{\text{digi}} - x_{\text{sim}}$:

```
python $CODE/analysis/digi_residuals.py hnl_digi.root --collection
    ↪ straw_tubes_hits
```

Fit the residual distribution. Does the width match the smearing in `gaussian_smearer.hpp`? Repeat for `timing_detector_hits`.

Exercise 15.3: Optional: make the straw model your own

Real straws measure the distance of closest approach to the wire with a resolution of roughly 100–150 μm, perpendicular to the wire only. In your **Shannon** checkout, give the straw digitiser its own resolution: in `src/detectors/straw_tubes.hpp` construct the smearer as `GaussianSmearer smearer_{0.12, 0.12};`, then

```
pixi run build
```

rerun the digitisation and the residual script, and check that only the straws changed. What would a realistic model need to know that a `SimHit` does not provide directly?

Chapter 16.

Analysis from the files

Where you are

Step: 10 of 11 in Part II.

You need: chapters 13 and 15: the simulated and digitised files; the Python of chapter 6.

You will be able to: select HNL decays from the files, weight them correctly, turn efficiencies into yields, and measure momentum from the bending.

There is no analysis framework to learn in this chapter. The files are RNTuples of plain structs, so a Python script with PyROOT can read them directly, and that is what we do.

16.1. Reading the files

The pattern from chapter 12 is all you need: open a reader on the RNTuple, create a typed *view* per field, and call the view with an entry number to get that entry's value as a C++ object. The scripts in `code/analysis/` share three small helpers in `shipfiles.py`: a class that keeps readers and views together, the detector numbering, and the positions of the decay volume and the straw stations.

code/analysis/shipfiles.py

```
class EventFile:
    """Typed, per-entry access to the fields of an RNTuple.

    >>> f = EventFile("hnl_sim.root")
    >>> f.view("mc_particles", "std::vector<SHiP::MCParticle>")
    >>> for i in f.entries(): particles = f["mc_particles"](i)
    """

    def __init__(self, path, ntuple="events"):
        self.path = path
        self.reader = ROOT.RNTupleReader.Open(ntuple, path)
        self.views = {}
        _alive.append(self.reader)

    def view(self, field, cpp_type, optional=False):
        """Create a view on `field`; with optional=True return None if absent.
    ↪ """
        try:
            v = self.reader.GetView[cpp_type](field)
        except Exception:
            if optional:
                return None
            raise
        _alive.append(v)
        self.views[field] = v
        return v

    def __getitem__(self, field):
        return self.views[field]
```

Pitfall

Two habits of PyROOT with RNTuples are built into these helpers. A view returns a reference into its own buffer, which the next call overwrites: copy out what you need before reading the next entry. And tearing down readers when Python exits can crash; the scripts therefore keep every reader alive and leave through `os._exit` after flushing their output (`sf.finish()`).

16.2. Which file for which question

Question	File
generator truth: HNL mass, decay position, daughter momenta	input or simulation
per event: which detectors did the daughters reach, was the veto silent	simulation
weighted efficiencies and yields	simulation (carries the weights)
detector response over all hits: resolutions, time distributions	digitisation

The reason for the split is the shape of **Shannon**'s output (section 15.4): a stream of time-stamped hits, as the detector will deliver, with no simulated-event number and no weights. Questions about one simulated event are therefore answered from the simulation file, using simulated hits as a stand-in for digitised ones. For the coarse station-count exercise, the position smearing is small. This does not establish equivalence for reconstructed tracks, vertex cuts, timing or a realistic detector response.

16.3. The HNL analysis

`code/analysis/hnl_analysis.py` answers, event by event:

1. What does the generator record say? It requires a pion and a muon of opposite electric charge with status 1, a shared status-2 mother and a common vertex, and fills the $\pi\mu$ invariant mass, the decay position and the opening angle.
2. Where did the daughters go? It finds them among the Geant4 particles (`parentId` $= 0$), and for each counts the straw stations in which it left hits.
3. Was the veto silent? It counts UBT hits and sums the SBT energy deposit.
4. How much did the magnet bend each daughter? From the mean x of its hits in stations 1–2 and 3–4 it computes the change of slope $\Delta\theta_x$ across the magnet.

It then applies these as a sequence of cuts, weighting every event with the EventCalc weight from `event_header`, and prints the cut flow.

Exercise 16.1: Run the analysis

From your working directory:

```
python $CODE/analysis/hnl_analysis.py hnl_sim.root
```

Look at the printed cut flow and at the PDFs it writes (`hnl_analysis_*.pdf`).

1. `_mass.pdf`: why is the peak so narrow, and where is it?
2. `_decay_z.pdf`: describe the shape. How would it change for a much shorter $c\tau$?
3. Which cut removes most of the weighted events? Look at `_n_stations.pdf` and `_opening.pdf` to find out why.

Checkpoint

A cut flow of seven lines whose first line counts all your simulated events, and eight PDFs.

Pitfall

The cut labelled "decay in nominal z range" checks only the longitudinal coordinate, not the helium volume or a fiducial margin. The UBT and SBT criteria also inherit the broad sensitive-volume selection. A full analysis needs a three-dimensional geometry check, active-material hit selection, reconstruction and background samples before quoting a sensitivity.

16.4. The veto

The SBT surrounds the decay volume to reject background from particles entering it from outside. Signal daughters should not touch it, but they can: a daughter emitted at a large angle, or a secondary from an interaction in the vessel or the first station, deposits energy in the LAB cells.

Exercise 16.2: What does a tight veto cost?

`_sbt_edep.pdf` shows the SBT energy deposit per event. Rerun with different thresholds and record the signal efficiency of the "SBT below threshold" line:

```
for t in 0 0.010 0.045 0.100; do
  python $CODE/analysis/hnl_analysis.py hnl_sim.root --sbt-threshold $t |
    ↪ grep SBT
done
```

A threshold costs signal efficiency here, and buys background rejection in a background sample. Why is the background side of this trade-off not something you can study with this sample?

16.5. Measuring momentum with the magnet

A charged particle crossing a dipole field changes direction by

$$\Delta\theta \approx \frac{0.3 \int B\,\mathrm{d}l\ [\mathrm{T\,m}]}{p\ [\mathrm{GeV}]},$$

for a singly charged, forward track in the small-angle limit. Thus $p\,|\Delta\theta_x|$ is approximately constant only for similar sampled field integrals and negligible energy loss and scattering. `_p_kick.pdf` plots it with the true momentum p and the $\Delta\theta_x$ measured from the simulated straw hits.

Exercise 16.3: The field integral from the hits

Read the position of the peak in `_p_kick.pdf` and convert it to $\int B\,\mathrm{d}l$. Compare with your estimate from the field map in section 10.2. If you simulated without field in chapter 13, run the analysis on that file too: where is the peak now?

16.6. Yields

The weights carry the probability of each decay; the absolute normalisation is EventCalc's expected number of decays N_{exp} (chapter 12). Provided its exposure, channel selection and sampled phase space match the input (with no omitted acceptance), it converts each weighted cut-flow fraction into a teaching yield estimate:

```
python $CODE/analysis/hnl_analysis.py hnl_sim.root --n-expected <N_exp>
```

Exercise 16.4: How many would SHiP see?

Take N_{exp} for the sample from **[TBD: where the converter or the organisers provide it]** and compute the expected number of selected $N \to \pi\mu$ decays. Which of the selection steps here would a real analysis replace, and with what?

16.7. Going further

Exercise 16.5: Optional: a vertex without reconstruction

Upstream of the magnet, straight lines approximate the daughter trajectories; material scattering and energy loss still matter. For each daughter, take the mean position of its simulated straw hits in stations 1 and 2, extrapolate the line back into the decay volume, and find the point of closest approach of the two lines. Histogram its distance from the true decay vertex in x and z. Why is the resolution so much worse in z than in x, and how does it depend on the opening angle?

Exercise 16.6: Optional: from a hit stream back to events

Using the digitised file, histogram the times of all straw hits. Could you separate the simulated events by time? What would the digitisation have to record so that a Monte Carlo study can connect digitised hits to the simulated event they came from? Bring your answer to the closing discussion.

Chapter 17.

Outlook: reconstruction with Trout

Where you are

Step: 11 of 11 in Part II.

You need: the rest of the booklet.

You will be able to: place reconstruction in the chain, and bring your questions about it to the closing discussion.

`ShipSoft/Trout` is the SHiP reconstruction. It is under active development and not part of the hands-on programme, but it closes the chain of figure 8.1, and its design answers several questions this tutorial raised. This chapter is the starting point for the closing discussion.

17.1. Where it stands

Trout is a set of Phlex plugins, like **aegir** and **Shannon**, and it depends on the same three packages you met first: the data model, the geometry and the field service. It runs on **Shannon** output or, later, on real detector data. Its main pieces:

`read_digitised_hits`
: reads **Shannon**'s per-detector hit streams. Grouping hits by time into candidate events is a later reconstruction step, not something the files impose.

`acts_geometry_provider`, `acts_field_provider`
: build the ACTS tracking geometry and the field, the latter through the field service's ACTS adapter.

`spectrometer_tracking`
: pattern recognition and fitting in three Phlex stages: per-spill preparation of the straw measurements, seeds from pairs of hits in stations 1 and 2, and for each seed a combinatorial Kalman filter (ACTS CKF) followed by a Kalman fit. Each seed is a separate Phlex data cell, so the framework schedules the fits in parallel.

detector modules
: for the UBT, SBT, timing detector and calorimeter.

`rec_output_module`
: writes `TrackFitResults` and reconstructed particles.

Vertexing is not there yet: the impact parameter in `RecParticle` is computed with respect to the origin as a placeholder.

17.2. Questions for the discussion

1. **From hits to events.** **Shannon** writes hit streams and **Trout** will form events by time. How should Monte Carlo truth (which simulated event, which particle, which weight) travel along so that efficiencies can be measured after reconstruction?

2. **The sensitive-volume contract. aegir** numbers detectors by position in a list, **Shannon** by an enum. Should the geometry itself tag sensitive volumes and their detector, so that the list in `hnl_sim.jsonnet` disappears?

3. **Digitisation models.** Every detector uses the same placeholder smearing. Which detector needs a realistic model first for the HNL analysis to be meaningful, and what does it need from the geometry service?

4. **Vertexing and the HNL selection.** The exercise in chapter 16 found a vertex from straight lines. What does a real vertex fit need, and where should it live?

5. **Versions.** One data-model change touches every package. How should the packages move together, and how should analysis scripts cope with member renames?

Appendix A.

Command reference

Commands assume the environment of the package named in the heading (`pixi shell` in its checkout), `work/` as the current directory except Geometry build commands, which run in its checkout, and `$CODE` pointing to `framework-booklet/code` in the supplied archive.

git

(chapters 1 to 3; full table in section 3.4)

```
git switch main && git pull upstream main          # start from the latest
    ↪ main
git switch -c task-NN-description                  # one branch per change
git add FILE && git commit -m "feat(scope): summary"
git push -u origin task-NN-description             # then open the pull
    ↪ request
git fetch upstream && git rebase upstream/main     # PR out of date
git push --force-with-lease                        # after a rebase (your
    ↪ branch only)
```

pixi

(chapter 4)

```
pixi init && pixi add python "root_base>=6.36"     # a new workspace
pixi install [--locked]                            # create the environment [
    ↪ exactly as locked]
pixi run <task or command>                         # one command in the
    ↪ environment
pixi shell                                         # an interactive shell in
    ↪ it; exit to leave
pixi task add <name> "<command>"                   # name a command
pixi run -e lint lint                              # a task in another
    ↪ environment
pixi update                                        # newer versions: commit
    ↪ the new pixi.lock
```

CI

(chapter 5)

```
pixi run --locked <task>                           # reproduce a CI step
    ↪ locally
pixi run lint                                      # what "Lint passed" checks
    ↪ , before you push
```

Team analysis

(chapter 6, in the team repository)

```
pixi run check-task NN                                  # test your task alone
pixi run check                                          # test every task (as CI
    ↪ does)
pixi run analysis                                       # run the chain as far as
    ↪ it goes
```

Geometry

(ShipSoft/Geometry)

```
pixi run geometry                                       # -> build/ship_geometry.db
pixi run ./build/apps/validate_geometry build/ship_geometry.db
pixi run gmex build/ship_geometry.db                                        #
    ↪ interactive viewer
pixi run gdml                                           # -> build/ship_geometry.
    ↪ gdml
```

Field maps

(ShipSoft/field_service)

```
fieldmap_dump MAP.root                                  # metadata
echo "0 0 0" | fieldmap_dump MAP.root NAME              # evaluate at points (mm ->
    ↪  T)
plot_field_map MAP.root NAME plots.root                 # B_y plots, ROOT + PDF
generate_constant_fieldmap OUT.root NAME Bx By Bz xmin xmax ymin ymax zmin
    ↪ zmax
fairship_to_fieldmap [--symmetry quadrant_dipole] IN.root OUT.root
```

Input check

Use the frozen PyROOT/data-model environment.

```
python $CODE/analysis/check_input.py hnl_events.root
```

Simulation

(ShipSoft/aegir)

```
phlex -j 4 -c <(jsonnet --ext-str events=N --ext-str infile=hnl_events.root \
    --ext-str simout=hnl_sim.root --ext-str histo=hnl_sim_validation.root \
    [--tla-code seed=S] $CODE/workflows/hnl_sim.jsonnet)
python $CODE/analysis/sim_summary.py hnl_sim.root [other.root]
```

Event display

(ShipSoft/sea-cucumber, run in its checkout)

```
pixi run sc --geometry DB --data hnl_sim.root --view views/default.toml --
    ↪ event I
pixi run sc --geometry DB --inspect 4 [--inspect-match "*trackers*"]
pixi run geo-cache --geometry DB --view views/default.toml
pixi run web-data --geometry DB --data hnl_sim.root --view views/default.toml
    ↪  --events all
pixi run web                                          # http://localhost:8080
```

Digitisation

(ShipSoft/Shannon)

```
phlex -j 4 -c <(jsonnet --ext-str entries=N --ext-str infile=hnl_sim.root \
    --ext-str digiout=hnl_digi.root --ext-str histo=hnl_digi_validation.root
    ↪ \
    $CODE/workflows/hnl_digi.jsonnet)
python $CODE/analysis/digi_residuals.py hnl_digi.root --collection
    ↪ straw_tubes_hits
```

Analysis

```
python $CODE/analysis/hnl_analysis.py hnl_sim.root [--n-expected N] [--sbt-
    ↪ threshold GeV]
python $CODE/analysis/solutions/vertex_from_hits.py hnl_sim.root
```

Looking inside any file

```
import ROOT
r = ROOT.RNTupleReader.Open("events", "hnl_sim.root")    # or "
    ↪ straw_tubes_hits", ...
r.PrintInfo(); r.Show(0)
```

Appendix B.

Data-model reference

Members of the classes in `ShipSoft/data-model`, in namespace `SHiP`. The first column gives the name in version 0.5, used throughout this booklet; the second gives the corresponding 0.6 name. Not every member changed. The compatibility limitations in chapter 11 still apply. Positions are in mm, momenta in GeV/c, energies in GeV, times in ns. Arrays of three are (x, y, z).

EventHeader

0.5	from 0.6	type	meaning
`weight`	`weight`	double	event weight; for EventCalc samples P_{decay}
`original_event_id`	`original_event_id`	int64	event number in the original generator file; -1 if unknown

MCParticle

0.5	from 0.6	type	meaning
`pdgCode`	`pdg_code`	int32	PDG particle code
`vertex`	`vertex`	double[3]	production vertex
`momentum`	`momentum`	double[3]	momentum
`energy`	`energy`	double	**total** energy
`time`	`time`	double	production time
`motherId`	`mother_id`	int32	index of the first mother in the same vector; -1 for none
`mothers`	`mothers`	vector<int32>	indices of all mothers (from 0.5)
`status`	`status`	int32	HepMC status; 1 = tracked by Geant4

SimParticle

0.5	from 0.6	type	meaning
`trackId`	`track_id`	int32	Geant4 track ID, unique within an event
`parentId`	`parent_id`	int32	track ID of the creator; 0 for generator particles
`pdgCode`	`pdg_code`	int32	PDG particle code
`vertex`	`vertex`	double[3]	production vertex
`endpoint`	`endpoint`	double[3]	where tracking stopped
`momentum`	`momentum`	double[3]	initial momentum
`energy`	`energy`	double	initial **kinetic** energy
`time`	`time`	double	production time
`creatorProcess`	`creator_process`	int32	Geant4 process sub-type that created the track

SimHit

0.5	from 0.6	type	meaning
`detectorId`	`detector_id`	int32	0 UBT, 1 SBT, 2 straws, 3 calorimeter, 4 timing
`geometryNodeId`	`geometry_node_id`	int32	identifier of the volume hit
`trackId`	`track_id`	int32	Geant4 track that made the hit
`pdgCode`	`pdg_code`	int32	PDG code of that track
`position`	`position`	double[3]	hit position
`momentum`	`momentum`	double[3]	momentum at the hit
`energyDeposit`	`energy_deposit`	double	energy deposited in the step
`time`	`time`	double	global time
`pathLength`	`path_length`	double	step length

RecHit, and the detector hits wrapping it

0.5	from 0.6	type	meaning
`detectorId`	`detector_id`	int32	as in `SimHit`
`trackId`	`track_id`	int32	truth link, as in `SimHit`
`pdgCode`	`pdg_code`	int32	truth link, as in `SimHit`
`position`	`position`	double[3]	measured position
`momentum`	`momentum`	double[3]	as in `SimHit`
`energyDeposit`	`energy_deposit`	double	as in `SimHit`
`time`	`time`	double	measured time
`pathLength`	`path_length`	double	as in `SimHit`
`UBTHit`, `SBTHit`, `StrawTubesHit`, `CaloHit`, `TimeDetHit`: one member `recHit` (0.6: `rec_hit`).			

RecParticle (reconstruction)

0.5	from 0.6	type	meaning
`trackId`	`track_id`	int32	
`parentId`	`parent_id`	int32	
`pdgCode`	`pdg_code`	int32	
`vertex`	`vertex`	double[3]	production vertex
`endpoint`	`endpoint`	double[3]	end point
`momentum`	`momentum`	double[3]	initial momentum
`hits`	`hits`	vector<RecHit>	hits on the particle
`energy`	`energy`	double	initial kinetic energy
`time`	`time`	double	production time
`creatorProcess`	`creator_process`	int32	
`ipPV`	`ip_pv`	double	impact parameter w.r.t. the primary vertex (now: the origin)

TrackFitResult (reconstruction)

0.5	from 0.6	type	meaning
`nMeas`	`n_meas`	int32	number of measurements
`fitStatus`	`fit_status`	int32	0 success, 1 failure
`chi2`	`chi2`	double	
`ndf`	`ndf`	int32	
`qoverp`	`q_over_p`	double	charge over momentum
`phi`	`phi`	double	
`theta`	`theta`	double	
`time`	`time`	double	
`refLoc`	`ref_loc`	double[3]	reference location
plus input and fitted measurements and residuals in x and y (vectors).			

Appendix C.

Troubleshooting

`Cannot locate geometry DB 'ship_geometry.db'; set SHIPGEOMETRY_ROOT ...`

aegir found the file neither in the directory you ran `phlex` from nor in the geometry search directory under `$SHIPGEOMETRY_ROOT`. Run from `work/`, or give an absolute path in the workflow.

`<file>: not a SHiP field-map file: ...`

The field-map file was not found or is not in the SHiP format. Bare names are looked up in the current directory, then in `$SHIPFIELD_ROOT/share/field/`. Check the `file` and `map` names with `fieldmap_dump`; convert FairShip maps with `fairship_to_fieldmap`.

`Field region '...': volume_pattern '...' matches no logical volumes`

A field `volume_pattern` matches no volume in the geometry. Patterns are substrings of the `/SHiP/...` names (section 9.2).

`No digitiser registered for detector ID n`

The simulation's `sensitive_volumes` list has more than five entries (section 13.3). A wrong order within IDs 0–4 does not raise this error, so verify the mapping explicitly. Correct the list and re-simulate.

`file_source: '...' has no 'event_header' field`

The input has no weights; **aegir** uses 1.0 for every event. Fix the converter for weighted yields; use unweighted files only for explicitly labelled shape or interface checks.

`AttributeError: ... has no attribute 'pdgCode'`

Your environment has data model 0.6 or later, where members are snake_case (appendix B). Use the matching environment, or rename in the script.

`Python crashes at the very end of a script (segmentation violation in RNTupleReader)`

A known PyROOT teardown issue. End scripts with `sf.finish()` (or `sys.stdout.flush(); os._exit(0)`), as the booklet's scripts do.

`jsonnet: RUNTIME ERROR: undefined external variable: events`

An `--ext-str` is missing from the `jsonnet` command. Run the `jsonnet` part alone to see what `phlex` would receive.

`phlex: cannot find plugin ...`

`PHLEX_PLUGIN_PATH` does not include the package's `build/` directory: you are not in that package's `pixi shell`, or the package is not built (`pixi run build`).

`The event display is empty or very slow`

Check `--data` points to the simulation file, not the digitised one. For speed, use a geometry cache (chapter 14) and keep the dense subsystems excluded in the view file.

`The web display shows nothing`

`pixi run web-data` must run before `pixi run web`, and the page needs internet access to load three.js.

Appendix D.

Review scope and source revisions

This review checks the supplied booklet against public repository source snapshots retrieved on 15 October 2026. The revisions below identify the reviewed code; they are *not* a jointly tested software release. The tutorial remains on the data-model 0.5 API. Development-head 0.6 changes are discussed explicitly rather than silently mixed into that workflow.

D.1. Checked primary sources

The revision links open the exact source trees used for the review.

Repository	Revision	Main checks
EventCalc	`7c9be2ad88`	Launch-card schema, event weights and coordinate convention
Geometry	`32dde5e213`	Coordinates, volume names, subsystem envelopes and tasks
Shannon	`908bccf8ba`	Input layout, smearing, spill seeds and per-object output
Trout	`5c0d1e4c33`	Hit-stream input and reconstruction scope
aegir	`43d0a32a60`	Input, sensitive-volume matching, scoring, field and output plugins
data-model	`09fb209fbb`	0.6 API changes and backward-read exceptions
field_service	`474d802850`	Dependency constraints, map I/O, interpolation and tools
geometry_service	`5afd27373e`	Geometry interface, lazy conversion and shared service
sea-cucumber	`bb2b0bbf64`	CLI, view rules, entry indexing and geometry backend
.github	`a788c399d8`	Branch rulesets, merge methods and required checks (section 3.3)

Particularly relevant files are `data-model/tests/data/README.md` (schema migration exceptions), the packages' `pixi.toml` manifests, `aegir/src/detector_construction.hpp` (substring-to-detector mapping), `aegir/src/geant4_sim_core.hpp` (positive-deposit scoring), `Shannon/src/digitise_hits.cpp` (random streams), and the EventCalc README's event-format description. Full commit IDs are included in `reviewed_sources.json`; the accompanying `REVIEW.md` records changes and validation limits.

D.2. Before the tutorial session

1. Freeze compatible package revisions and lock files. Resolve the field-service 0.6 versus aegir 0.5 dependency conflict; pin the display to a compatible data model. Verify all packages on the intended machines.
2. Provide the HNL input and converter with their mass, lifetime, channel, exposure, frame transform, mother links and weight convention. Preserve `original_event_id` and the expected-yield metadata.
3. Provide field maps, their names and coordinate placements. Check the maps against the exact geometry database used by all consumers.
4. Review the five detector selectors. Broad subsystem matching includes passive material and cannot support physical veto efficiencies or calorimeter response without an active-volume mapping or filter.

5. Run the full chain on the frozen sample, verify schema readback in each environment, and replace the remaining **TBD** values with measured results (including runtime and disk requirements).

D.3. Accessibility and validation

The booklet uses dark blue and neutral tones on white, with dark text and light box backgrounds. Box titles state their meaning; arrows distinguish data, services and the future branch by solid, dashed and dotted strokes. No task or warning requires distinguishing hues. The supplied black SHiP logo is included without recolouring. Analysis plots use a high-contrast single-series line; Gaussian fits additionally use a black dashed line. This supports greyscale use and avoids dependence on red–green perception; it is not a claim of formal PDF accessibility certification.

Both editions are compiled and visually checked. Python files are checked for syntax, and numerical/selection helpers are exercised with synthetic inputs. The EventCalc card and workflow structures are checked separately. No full Geant4 or digitisation run is claimed: the supplied archive has no ROOT event sample, maps or frozen integration environment.

For Part I, the script `tests/test_git_workflow.sh` runs every git command sequence of chapters 1 to 3 and 6 against local bare repositories standing in for GitHub (forks, an out-of-date pull request, a rebase with and without conflict, a rebase-only merge queue). The team exercise (skeleton, per-task checker, sample generator and reference solutions) runs end to end in `tests/test_teamwork.py` against a numerical stand-in for PyROOT (`tests/fakeroot/`); it has not yet been run with real ROOT. The pixi manifests and the task, feature and environment commands of chapter 4 were checked with pixi 0.82 on workspaces without packages (the package channels were not reachable from the review environment, so no environment was solved and no lock file for the team repository exists yet); both workflows of chapter 5 pass actionlint 1.7.12, and all tutorial Python passes the ruff configuration CI uses. None of the workflows has run on GitHub.